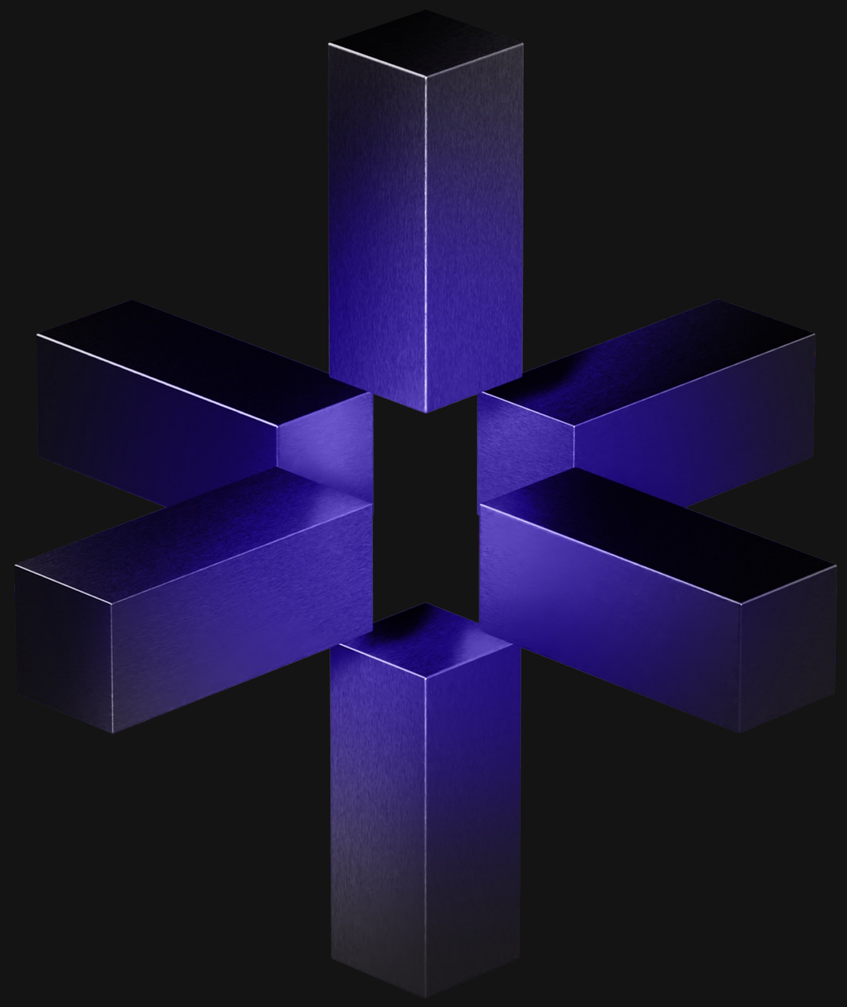
ЦЕНТРАЛЬНЫЙ УНИВЕРСИТЕТ



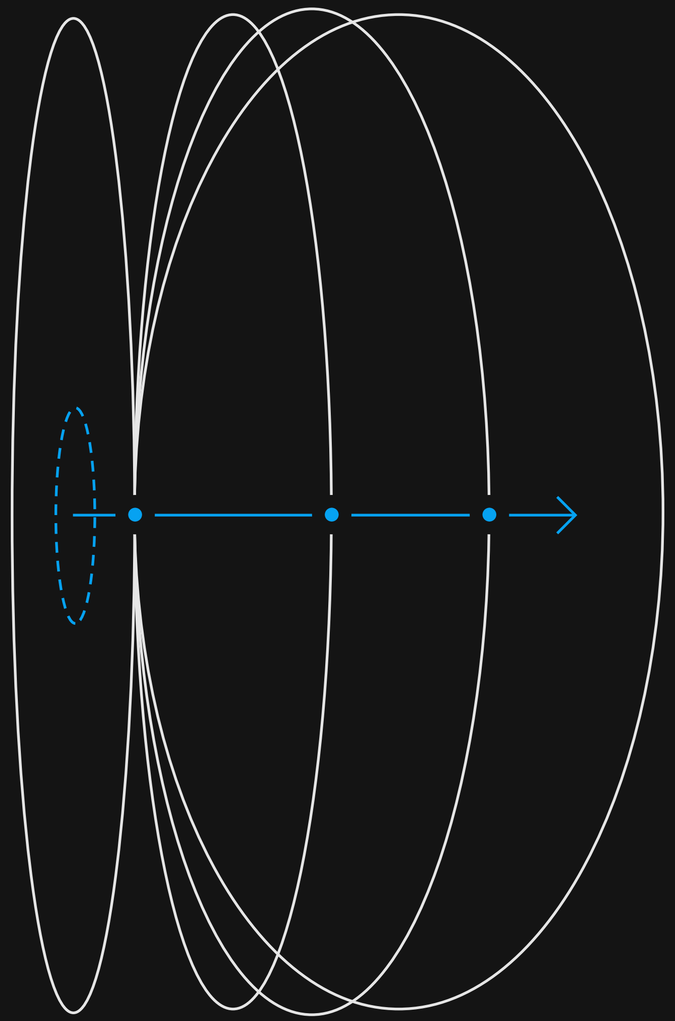
Логистическая регрессия и SVM

ML

Машинное обучение

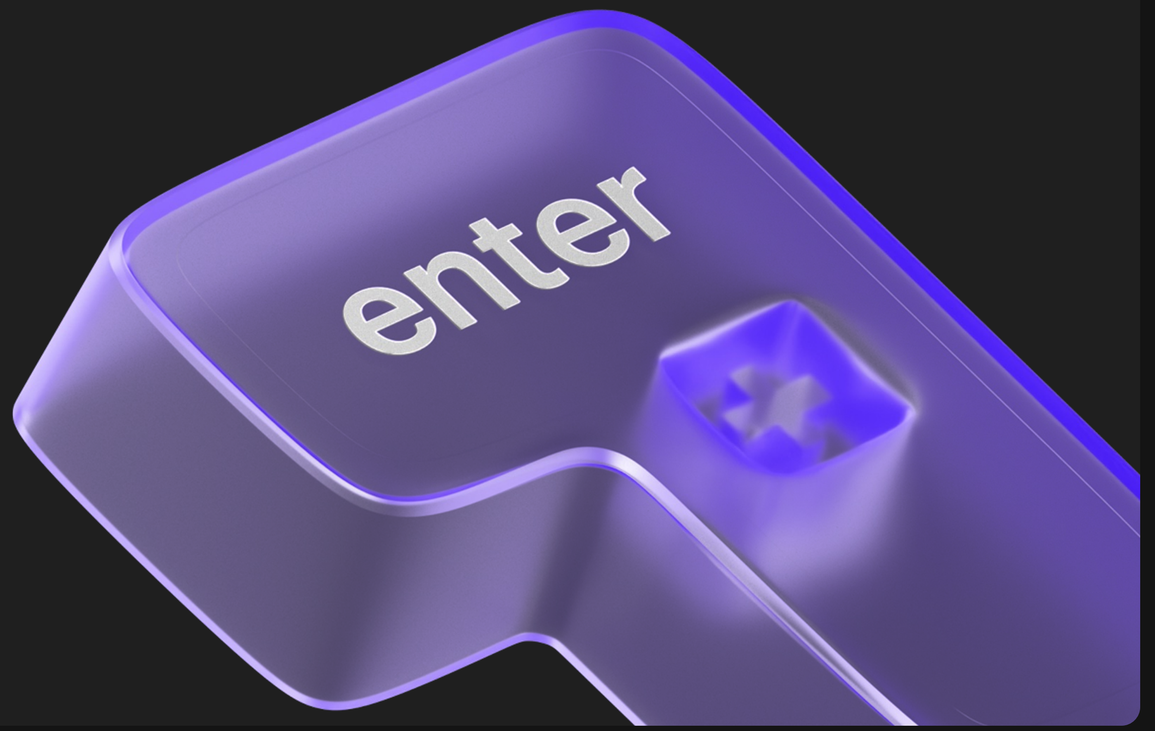
05

План лекции



- Задача классификации
- Логистическая регрессия
- Метод опорных векторов (SVM)
- Различия SVM и логистической регрессии
- Калибровка вероятностей
- Преобразование признаков и целевой переменной

Задача классификации



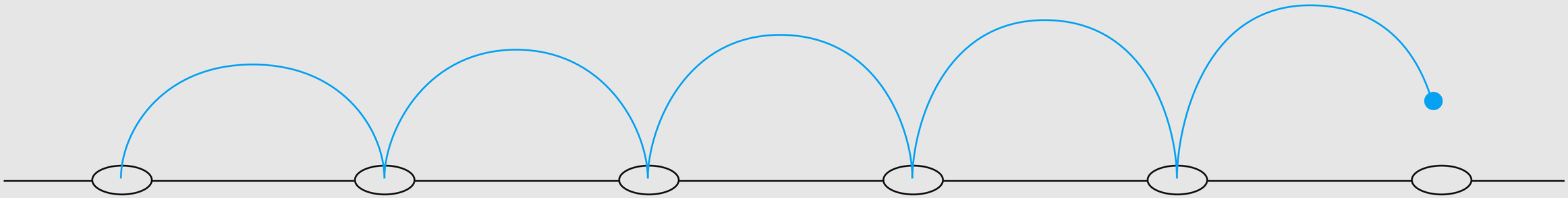
Классификация

Классификация — это задача машинного обучения, в которой алгоритм учится относить объекты к одному из заранее определённых классов (категорий) на основе их признаков.

Задача классификации — предсказать класс объекта, а не число (в отличие от регрессии).

Примеры

- Спам или важное письмо?
- Пациент болен гриппом или простудой?
- Банк одобрит заявку на кредит или пришлёт отказ?



Классификация

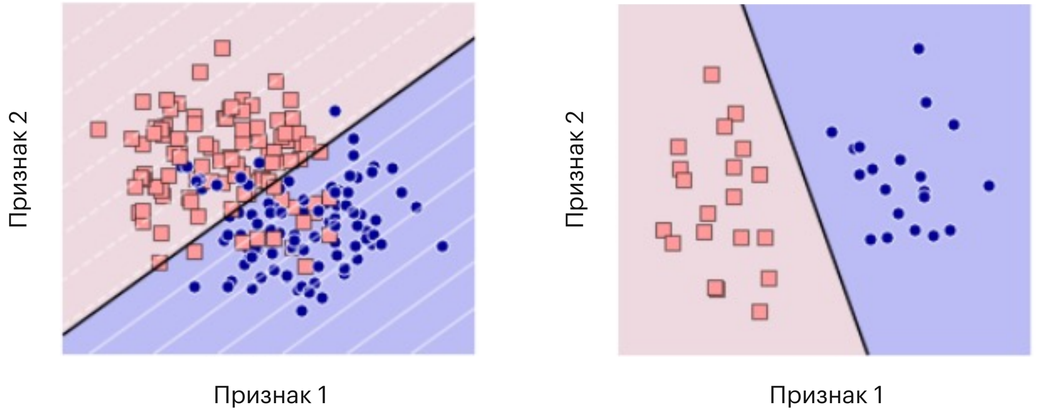
Для бинарной классификации целевая переменная принимает значения:

$$y \in \{-1, +1\}$$

Где:

- $y = +1$ — объект, который принадлежит положительному классу (например, «одобрить»);
- $y = -1$ — объект, который принадлежит отрицательному классу (например, «отказать»).

Интерпретация: делим пространство гиперплоскостью на две части.



Как модель проводит границу

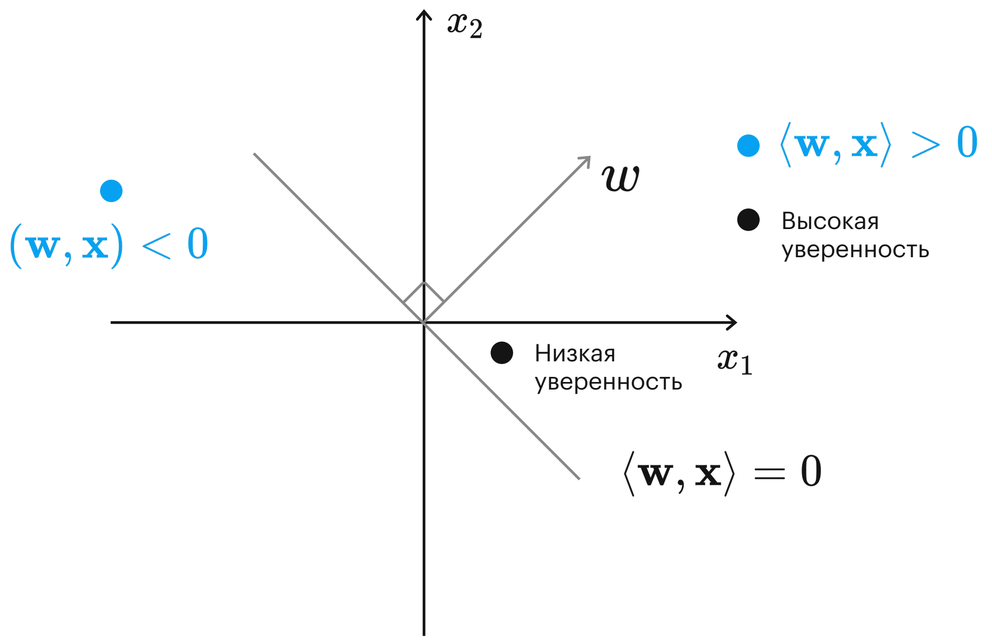
Решающее правило:

$$a(x) = sign(\langle w, x \rangle + b)$$

Где:

w — вектор весов, который определяет направление разделяющей границы и то, насколько резко модель переходит от одного класса к другому;

b — смещение гиперплоскости, сдвигающее её относительно начала координат.



$$a(x) = \begin{cases} +1, & \text{если } \langle \mathbf{w}, \mathbf{x} \rangle + b > 0, \\ -1, & \text{если } \langle \mathbf{w}, \mathbf{x} \rangle + b < 0. \end{cases}$$

Отступ (Margin)

M — величина, которая показывает, насколько уверенно модель классифицирует объект.

Определяется формулой

$$M(\mathbf{x}, y) = y \cdot (\langle \mathbf{w}, \mathbf{x} \rangle + b)$$

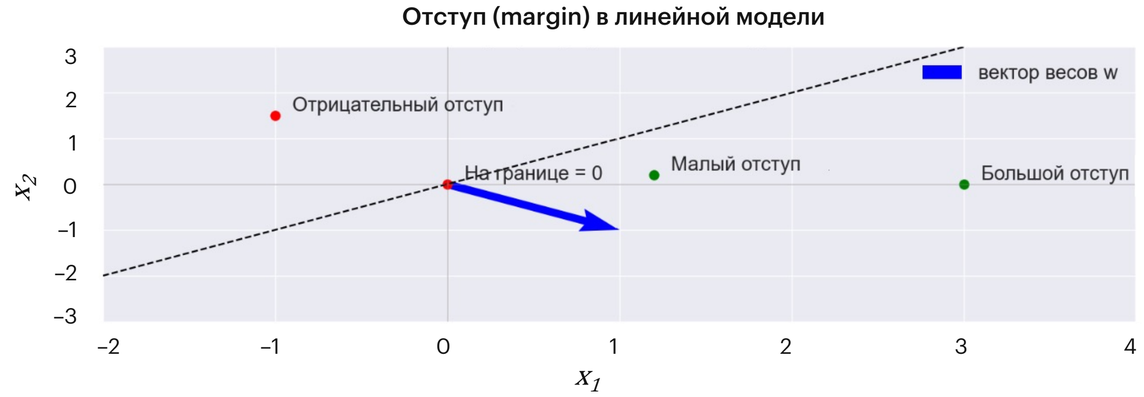
где:

* $y \in \{-1, +1\}$ — истинная метка объекта.

* $\{w, x\} + b$ — сырой выход из модели.
После этого значения обычно применяется функция sign или другая активация для получения предсказания класса.

Мы умножаем «сырое» предсказание модели на правильный ответ. Если:

- $M > 0$, то модель права;
- $M < 0$, то модель ошибается;
- $M = 0$, то объект лежит на границе (неопределённость).



Функция ошибок и её ограничения

Если модель ошиблась, то $M_i \leq 0$. Количество ошибок при этом можно посчитать с помощью формулы:

$$Q(w, b) = \frac{1}{n}\sum_{i=1}^{n}[M_i \leq 0] = \frac{1}{n}\sum_{i=1}^{n}[y_i(\langle w, x_i\rangle + b) \leq 0]$$

где:

$M_i = y_i(\langle w, x_i\rangle + b)$ — отступ;

То есть, мы считаем долю ошибок на всей выборке.

Функция ошибок и её ограничения

Если модель ошиблась, то $M_i \leq 0$. Количество ошибок при этом можно посчитать с помощью формулы:

$$Q(w,b) = \frac{1}{n}\sum_{i=1}^{n}[M_i \leq 0] = \frac{1}{n}\sum_{i=1}^{n}[y_i(\langle w, x_i\rangle + b) \leq 0]$$

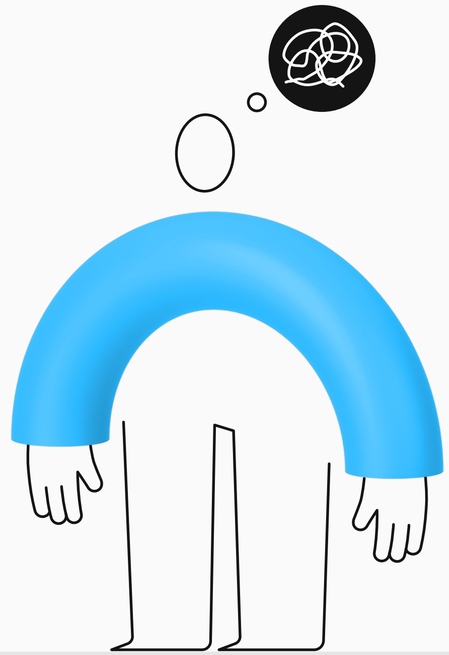
где:

$M_i = y_i(\langle w, x_i\rangle + b)$ — отступ;

То есть, мы считаем долю ошибок на всей выборке.

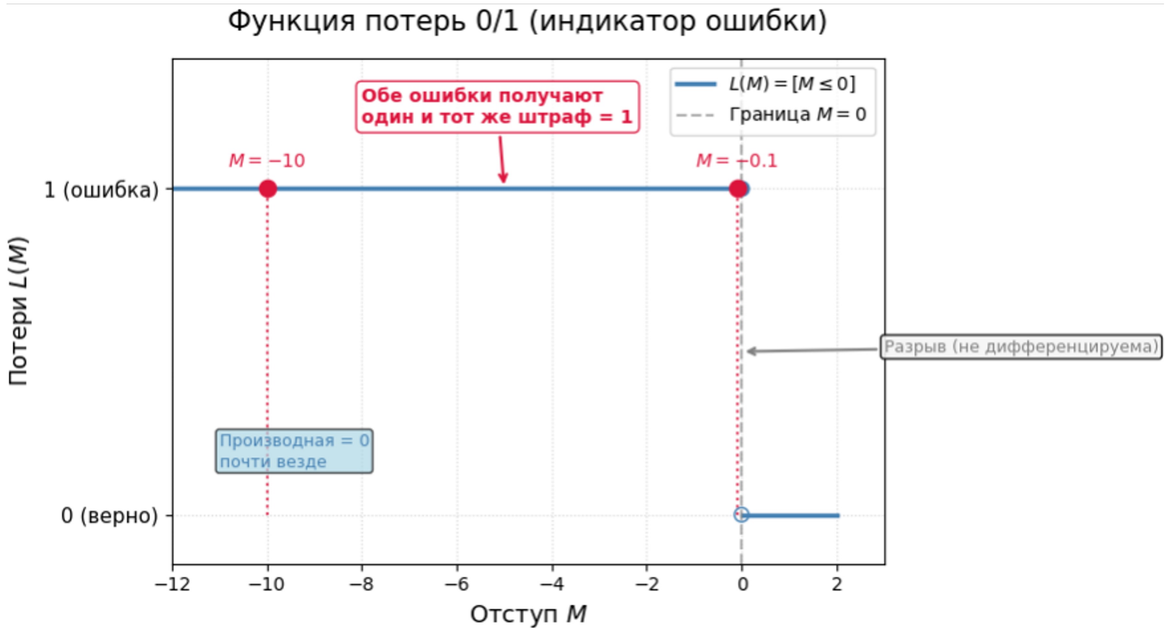
Вопрос

Какие можно назвать две критические проблемы такой функции ошибок?



Ограничения функции ошибок

- **Недифференцируема (кусочно-постоянная).** Градиент либо не определен, либо равен нулю.
- **Нечувствительна к степени ошибки.** Например, $M = -0.1$ и $M = -10$ штрафуются одинаково.

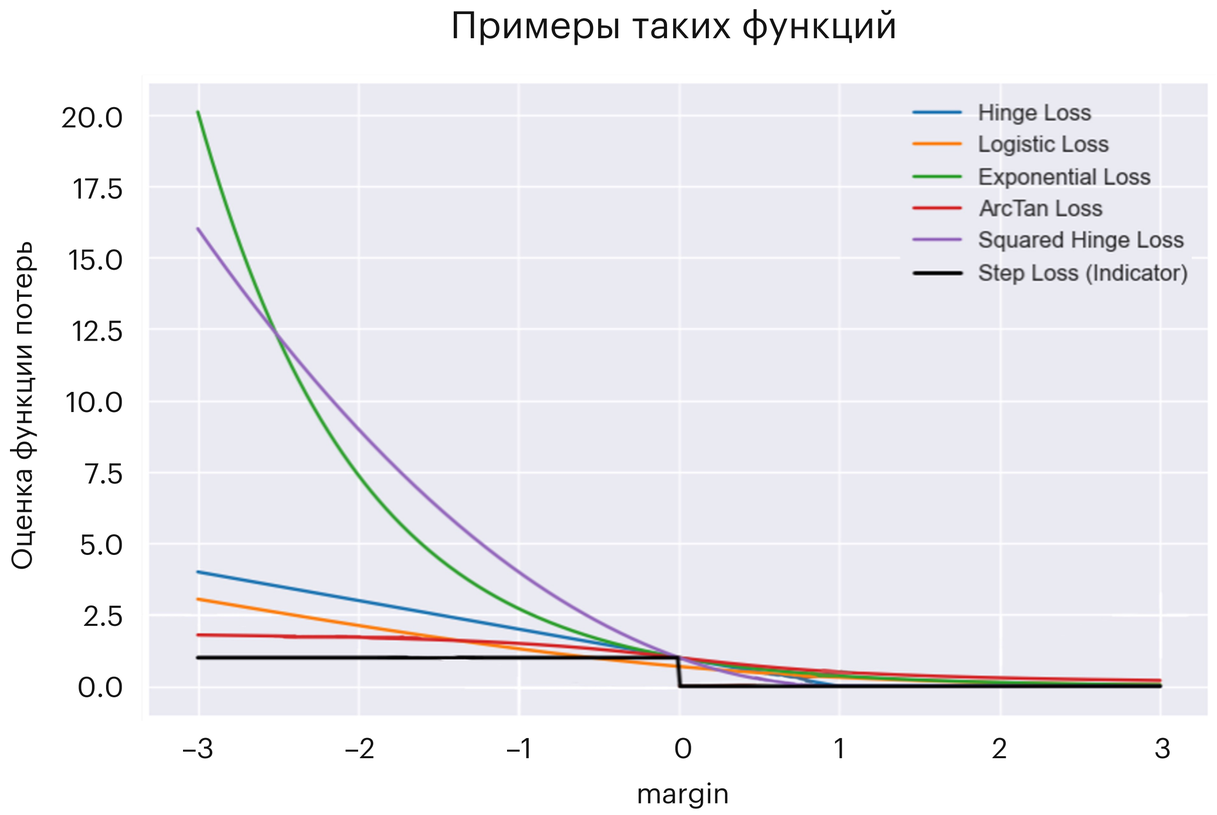


Аппроксимация функции потерь

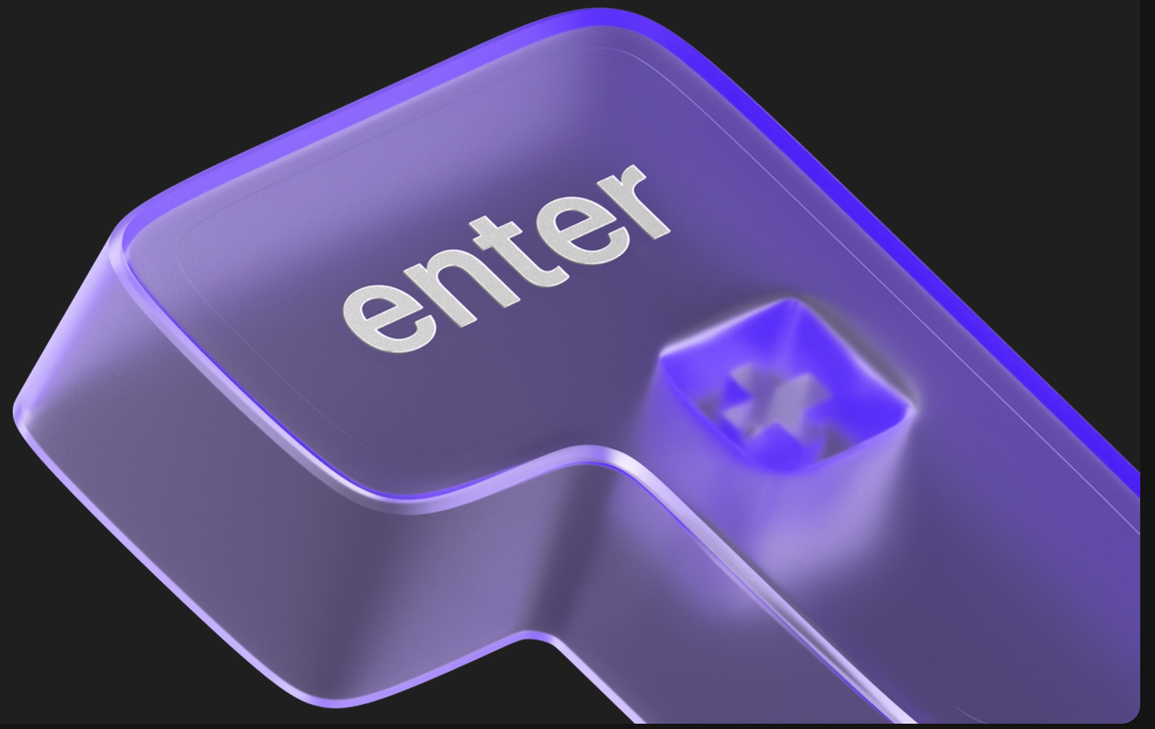
Классический приём в математике и машинном обучении — замена неудобной функции $[Mi \leq 0]$ на более удобную для обучения функцию потерь $L(Mi)$, которая:

- штрафует за ошибки;
- поощряет уверенные предсказания.

Такие функции удобнее использовать при обучении: некоторые из них гладкие, у других есть излом



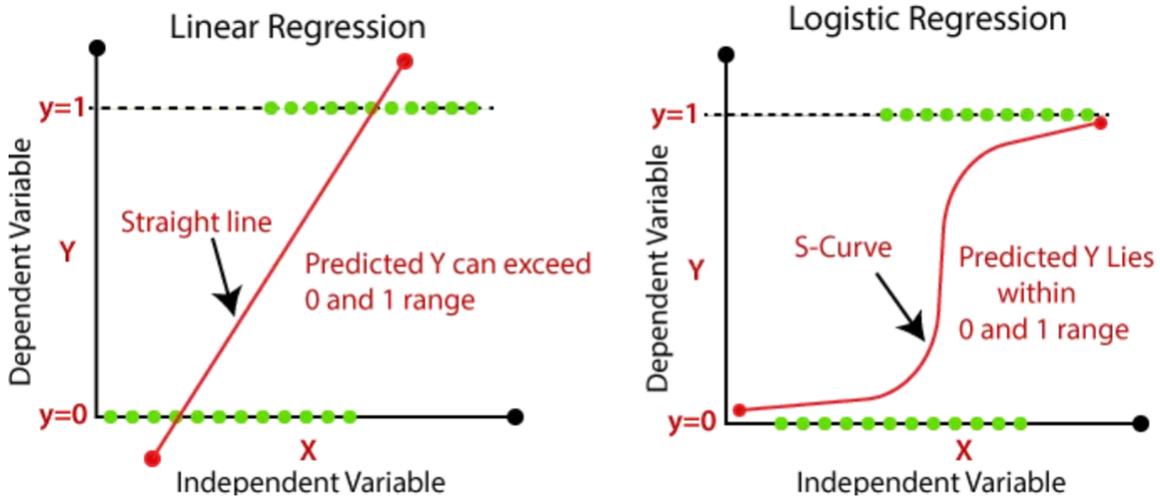
Логистическая регрессия



От линейной к логистической регрессии

Вопрос

На основе линейной регрессии кредитная модель рассмотрела заявки двух клиентов и вернула значения 1.8 и 0.9. Значит ли это, что риск дефолта первого клиента в два раза выше?



Источник

От линейной к логистической регрессии

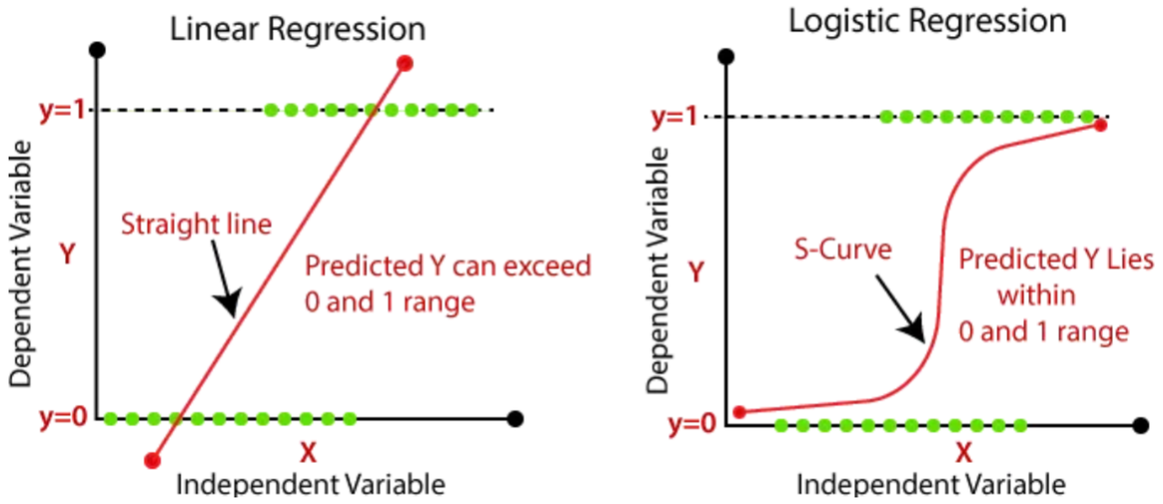
Вопрос

На основе линейной регрессии кредитная модель рассмотрела заявки двух клиентов и вернула значения 1.8 и 0.9. Значит ли это, что риск дефолта первого клиента в два раза выше?

Ответ

Нет. Выход линейной регрессии не является вероятностью и может выходить за пределы от 0 до 1.

Необходимо преобразовать выход линейной модели в число от 0 до 1, которое можно интерпретировать как вероятность



Источник

От линейной к логистической регрессии

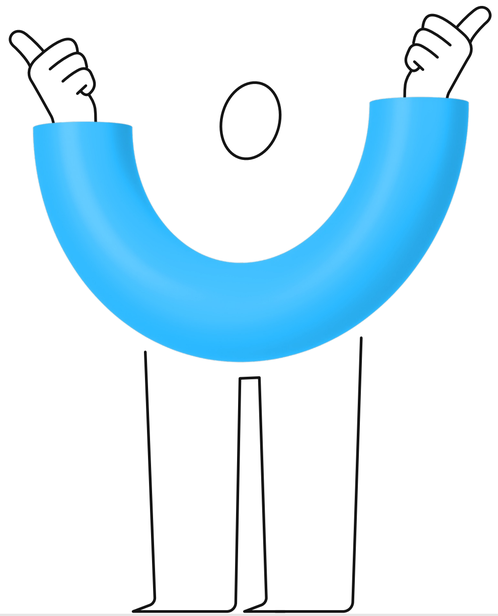
$a(x) = \text{sign}(\langle w, x \rangle + b)$

`sign` использует только знак сырого выхода и отбрасывает информацию о его величине.

Вопрос: как преобразовать $< w, x > + b$ в вероятность положительного класса?

Нужна функция, которая:

- возвращает число от 0 до 1;
- растёт вместе с $(w, x) + b$;
- равна 0.5, когда $(w, x) + b = 0$.



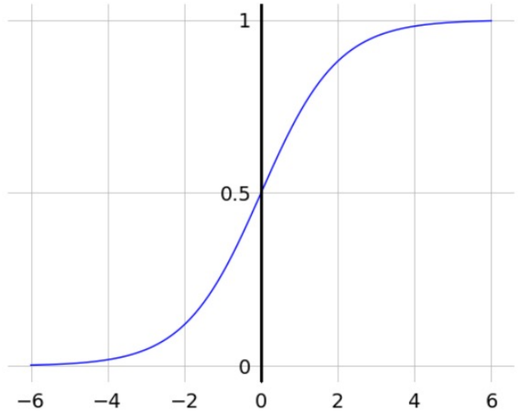
Сигмоидная функция: «сжиматель» в вероятности

Сигмоида удобна тем, что она всегда возвращает значения в интервале от 0 до 1, независимо от того, насколько велико z.

Ключевая формула:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Преобразует значение $z = \langle w, x \rangle + b$ в число из интервала $[0,1]$.



Свойство	Математически	Практический смысл
Диапазон	$\sigma(z) \in (0,1)$	Гарантирует корректную вероятность
Поведение на краях	$\sigma(z) \to 1$ при $z \to +\infty$ и $\sigma(z) \to 0$ при $z \to -\infty$	Уверенные предсказания при больших z
Точка перехода	$\sigma(0) = 0{,}5$	Неопределённость при $z = 0$

Применение сигмоиды

Формула вероятности положительного класса:

$$P(y = +1 \mid x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

где:

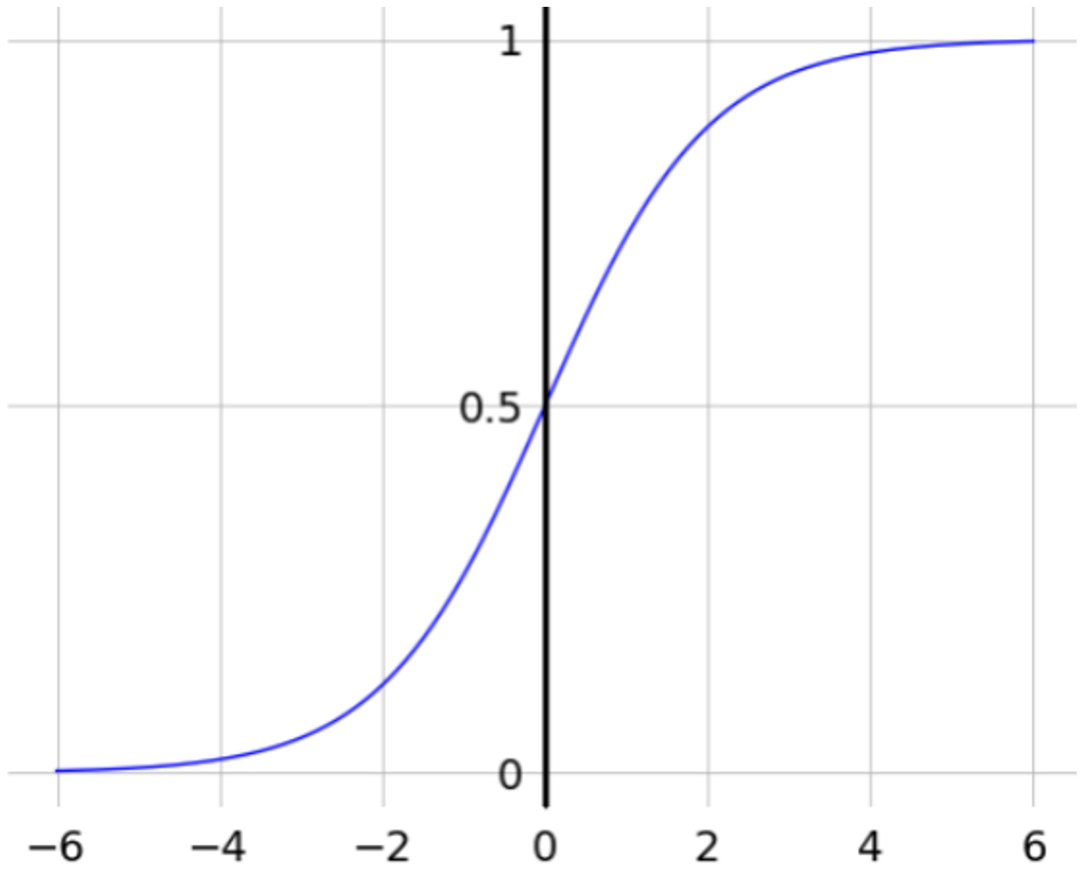
$z = \langle w, x \rangle + b$ — сырой выход линейной модели

При стандартном пороге 0.5:

$z < 0 \Rightarrow P(y = +1 \mid x) < 0.5 \Rightarrow \hat{y} = -1,$

$z \geq 0 \Rightarrow P(y = +1 \mid x) \geq 0.5 \Rightarrow \hat{y} = +1$

Выходит, что разделяющая граница $z = 0$ остаётся прежней, но теперь модель дополнительно выдаёт вероятность положительного класса.



Другой взгляд на сигмоиду

Другой способ прийти к логистической регрессии — предположить, что линейно от признаков зависит не вероятность p, а **логарифм отношения вероятностей двух классов**.

Отношение $\frac{p}{1-p}$ называют шансами. Например, при $p = 0.8$ оно равно 4 — положительный класс в 4 раза вероятнее отрицательного.

$$\log \frac{p}{1-p} = \langle w, x \rangle + b$$

Логарифм отношения вероятностей двух классов

Другой взгляд на сигмоиду

Таким образом, предположение о линейности логарифма шансов приводит к сигмоиде.

Подробнее об этом — в лонгриде.

Шансы

$$\text{odds} = \frac{P(y = +1 \mid x)}{P(y = -1 \mid x)} = \frac{p}{1 - p}$$

Логарифмируем и моделируем линейно

$$\log\left(\frac{p}{1 - p}\right) = \langle w, x \rangle + b$$

Делаем ряд преобразований

$$\frac{p}{1 - p} = e^{\langle w,x \rangle + b} \Rightarrow p = \frac{e^{\langle w,x \rangle + b}}{1 + e^{\langle w,x \rangle + b}} = \frac{1}{1 + e^{-(\langle w,x \rangle + b)}}$$

И получаем логистическую регрессию

$$P(y = +1 \mid x) = \sigma(\langle w, x \rangle + b)$$

ЦУ

Как подобрать параметры модели

Мы определили, как модель вычисляет вероятность:

$$p_i = P(y_i = +1 \mid x_i) = \sigma(\langle w, x_i \rangle + b)$$

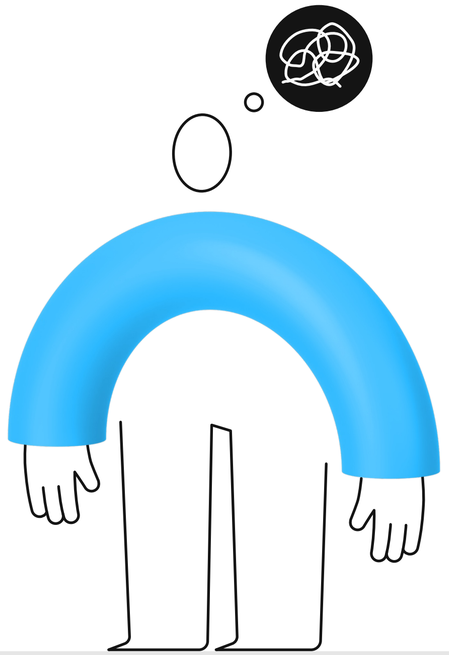
Теперь нужно подобрать w и b так, чтобы модель давала высокую вероятность наблюдаемому ответу:

$y_i = +1 \quad \Rightarrow \quad p_i$ должно быть большим,

$y_i = -1 \quad \Rightarrow \quad 1 - p_i$ должно быть большим.

Вопрос

Как объединить эти требования для всех объектов в один критерий обучения?



Максимизация правдоподобия: основа обучения

Цель обучения модели — найти параметры w, которые максимизируют вероятность (или правдоподобие) наблюдаемых данных.

Исходная функция правдоподобия:

$$\mathcal{L}(w) = \prod_{i=1}^{\ell} p(y_i \mid x_i, w)$$

Логарифмирование для устранения произведения:

$$\log \mathcal{L}(w) = \sum_{i=1}^{\ell} \log p(y_i \mid x_i, w)$$

Максимизация = минимизация отрицательного:

$$\mathcal{L}(w) = -\sum_{i=1}^{\ell} \log p(y_i \mid x_i, w) \to \min$$

Логарифмическая функция потерь (logloss)

Чтобы получить logloss, в логарифм подставляют вероятности, полученные через сигмоиду:

- Вероятность для класса +1:

  $$p(y = +1 \mid x) = \sigma(\langle w, x \rangle) = \frac{1}{1 + e^{-\langle w,x \rangle}}$$

- Вероятность для класса -1:

  $$p(y_i = -1 \mid x_i, w) = 1 - \sigma(\langle w, x_i \rangle)$$

$$\mathcal{L}(y_i, \hat{y}_i) = -([y_i = +1] \log \sigma(\langle w, x_i \rangle) + [y_i = -1] \log(1 - \sigma(\langle w, x_i \rangle)))$$

Объект принадлежит к классу +1

Объект принадлежит к классу -1

Логарифмическая функция потерь (logloss)

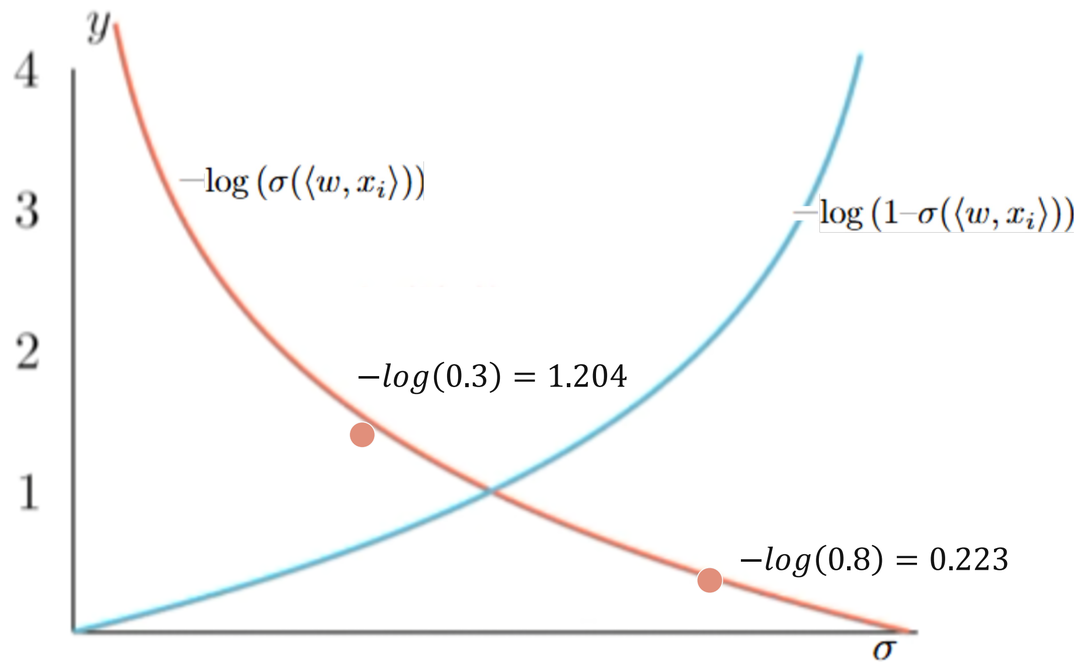
Синоптик А предсказал вероятность дождя в 80%, а синоптик Б — 30%.
В реальности дождь пошел.

Синоптик Б был самоуверенно неправ — и получил штраф почти в 5.4 раза больше.
То есть он не ожидал, что дождь пойдет. Таким образом, кросс-энтропия показывает “меру неожиданности” предсказания.

$$\mathcal{L}(y_i, \hat{y}_i) = -\Big([y_i = +1]\log\sigma(\langle w, x_i\rangle) + [y_i = -1]\log\big(1 - \sigma(\langle w, x_i\rangle)\big)\Big)$$

Объект принадлежит к классу +1

Объект принадлежит к классу -1



Логарифмическая функция потерь (logloss)

Чтобы получить logloss, в логарифм подставляют вероятности, полученные через сигмоиду:

- Вероятность для класса +1: $p(y = +1 \mid x) = \sigma(\langle w, x \rangle) = \frac{1}{1+e^{-\langle w,x \rangle}}$
- Вероятность для класса -1: $p(y_i = -1 \mid x_i, w) = 1 - \sigma(\langle w, x_i \rangle)$

Компактная запись для обоих классов (для одного наблюдения):

$$\mathcal{L}(y_i, \hat{y}_i) = -\big([y_i = +1] \log \sigma(\langle w, x_i \rangle) + [y_i = -1] \log\big(1 - \sigma(\langle w, x_i \rangle)\big)\big)$$

Финальный вид

$$\mathcal{L}(w) = \sum_{i=1}^{\ell} \log(1 + \exp(\underbrace{-y_i \langle w, x_i \rangle}_{\text{Margin}}))$$

Интерпретация предсказанных вероятностей

$$p(x) = P(y = +1 \mid x) = \sigma(\langle w, x \rangle + b)$$

Вероятность встроена в саму модель и её функцию потерь. Поэтому результат логистической регрессии сразу интерпретируется как вероятность положительного класса (при class_weight=None; при перевзвешивании классов калибровку вероятностей необходимо проверять отдельно).

Например, $p = 0.8$ означает, что среди большого числа объектов с такими же или похожими характеристиками:

- примерно 80% будут принадлежать классу +1;
- а 20% — классу -1

Логистическая регрессия:

- сразу выдает вероятность;
- результат лежит от 0 до 1;
- отдельная калибровка для получения вероятностного ответа не требуется.

Модели, выдающие скор (например SVM):

- выдают произвольное число или отступ;
- скор не является вероятностью;
- для перехода к вероятностям требуется калибровка.

Вызов в sklearn

```
from sklearn.linear_model import LogisticRegression

# Обучим базовую модель логистической регрессии.
model_logreg = LogisticRegression(
    penalty='l2',
    solver='liblinear',
    max_iter=1000,
    fit_intercept=True,
    class_weight='balanced'
)

# penalty — тип регуляризации; 'l2' — по умолчанию, 'l1' даёт разреженность (меньше признаков).
# solver — алгоритм обучения; нужен при 'l1'; иначе можно не менять.
# max_iter — число итераций до сходимости; нужно увеличивать при предупреждении ConvergenceWarning.
# fit_intercept — добавлять ли свободный член (смещение); True — по умолчанию
# class_weight — учитывает дисбаланс классов; полезно, если один класс встречается намного реже другого.
```

Вызов в sklearn

```
# Фит на данные
model_logreg.fit(X_train, y_train)

# Предсказания для теста
# Вероятности принадлежности к положительному классу (1)
y_pred_prob = model_logreg.predict_proba(X_test)[:, 1]

# Предсказания классов, а не вероятностей
y_pred_classes = model_logreg.predict(X_test)

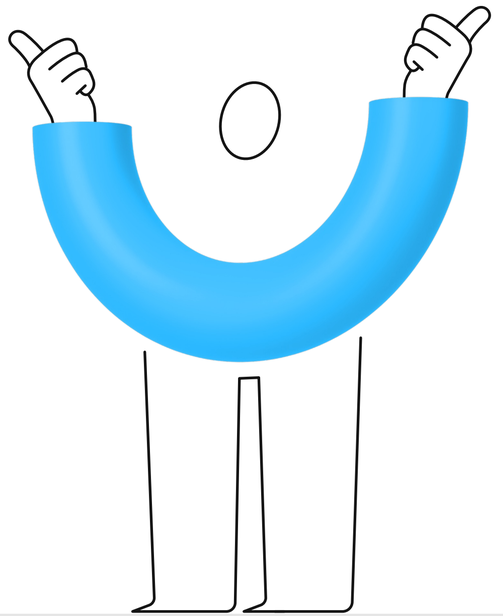
# Вывод коэффициентов модели
print(model_logreg.coef_)
print(model_logreg.intercept_)
```

Применение

Логистическая регрессия популярна в бизнесе. Особенно там, где важна интерпретируемость — например, при кредитном скоринге.

Преимущества логистической регрессии:

- предоставляет честные вероятности (по математическому выводу);
- помогает гибко настраивать порог бинаризации.



Применение: Пример 1

Пример. Пусть у нас есть модель для кредитного скоринга Сначала модель вычисляет линейный скор z, после чего преобразует его с помощью сигмоиды в вероятность дефолта:

$$z = w_1 DTI + w_2 CreditScore + w_3 \ln(Income) + w_4 PastDefaults + b$$

$$P(Default \mid x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Модель была обучена и получены следующие коэффициенты.

DTI	+2.3	Чем выше долговая нагрузка заемщика относительно его дохода, тем выше риск дефолта
CreditScore	-0.003	Более высокий кредитный рейтинг снижает риск дефолта
ln_Income	-0.25	Более высокий доход связан со снижением риска дефолта
PastDefaults	+1.2	Каждая дополнительная просрочка в кредитной истории существенно увеличивает риск
Свободный член b	-0.5	—

Применение: Пример 1

Модель можно использовать
как для интерпретаций, так и по прямому назначению. Например, имеем клиента с следующими характеристиками:

Возраст	32 года
Ежемесячный доход	85 000 ₽
Сумма кредита	600 000 ₽
Срок кредита	36 месяцев
Кредитный рейтинг	680
DTI (Debt to Income) Соотношение долга и дохода	0,42
Просрочки в прошлом	1

Оценим вероятность дефолта и стоимость риска.

1. Вероятность дефолта: $z = 2.3 \cdot 0.42 + (-0.003) \cdot 680 \; + \\ + (-0.25) \cdot \ln(85000) + 1.2 \cdot 1 + (-0.5) = -3.211$

$$p(\text{дефолт}) = \frac{1}{1 + e^{-(-3.211)}} \approx 0.039$$

2. Используя эту вероятность, банк может оценить стоимость риска — сколько надо денег заложить на резервы в случае дефолта клиента.
   При предположении, что в случае дефолта вся сумма кредита будет потеряна, ожидаемые потери равны:

$$600000 \times 0.039 = 23400 \text{ ₽}$$

Применение: Пример 2

Логистическая регрессия популярна в бизнесе, особенно там, где важна интерпретируемость (например, при кредитном скоринге).

Её преимущества:

- предоставляет честные вероятности,
- позволяет гибко настраивать порог.

Пример. Модель, используемая в онлайн-рекламе, предсказывает вероятность клика для каждого баннера по формуле: мат. ожидание прибыли = вероятность клика × доход с одного клика.

1. Допустим, баннер A: p = 0.1, доход за клик = 100 → $\mathbb{E}$[прибыль] = 0.1×100 = 10.
2. Баннер B: p = 0.02, доход за клик = 1000 → $\mathbb{E}$[прибыль] = 0.02×1000 = 20.
3. Баннер C: p = 0.5, доход за клик = 10 → $\mathbb{E}$[прибыль] = 0.5×10 = 5.

Результат: показываем баннер B. Он показывает наибольшее ожидаемое значение прибыли.

Итоги логистической регрессии

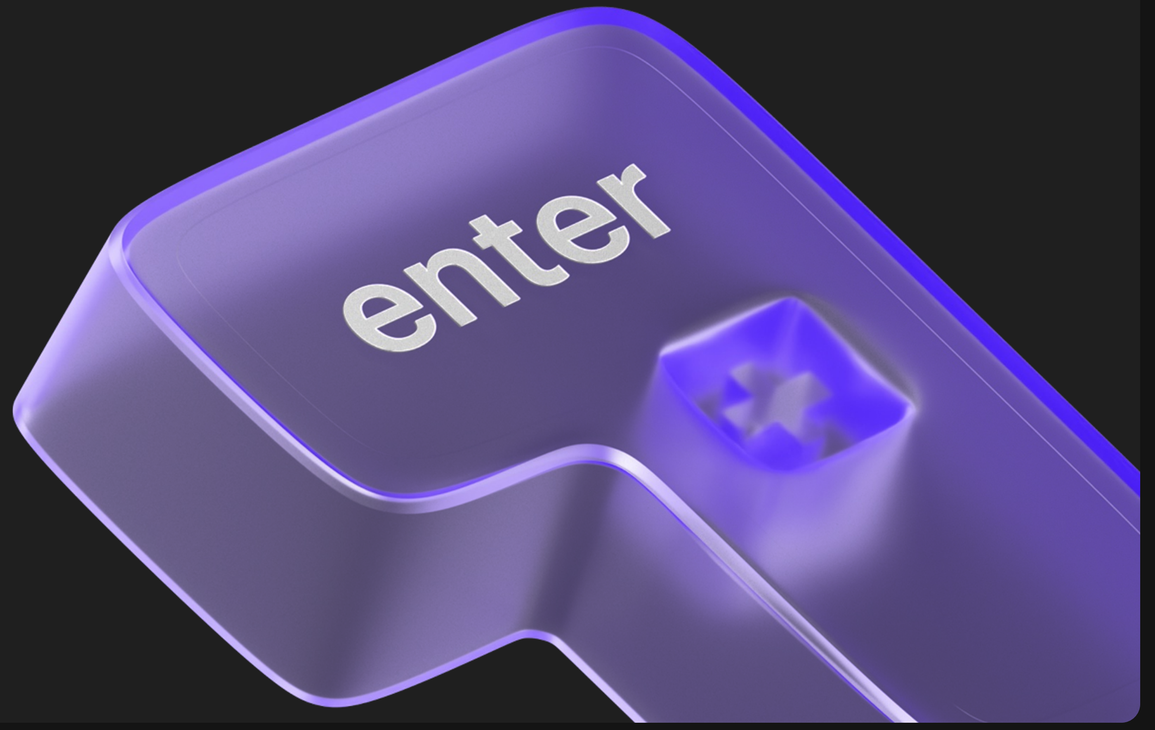
Логистическая регрессия — модель бинарной классификации, которая превращает линейную комбинацию признаков в вероятность положительного класса при помощи сигмоидной функции: $a(x) = P(y = +1 \mid x) \in [0,1]$

Обучение модели основано на максимизации правдоподобия (MLE), что эквивалентно минимизации логистической функции потерь (она же бинарная кросс-энтропия). Задача выпуклая и решается градиентными методами.

Модель проста, вычислительно эффективна и интерпретируема, поэтому широко применяется в кредитном скоринге, риск-менеджменте и других областях, где важна объяснимость и соблюдение регуляторных стандартов.

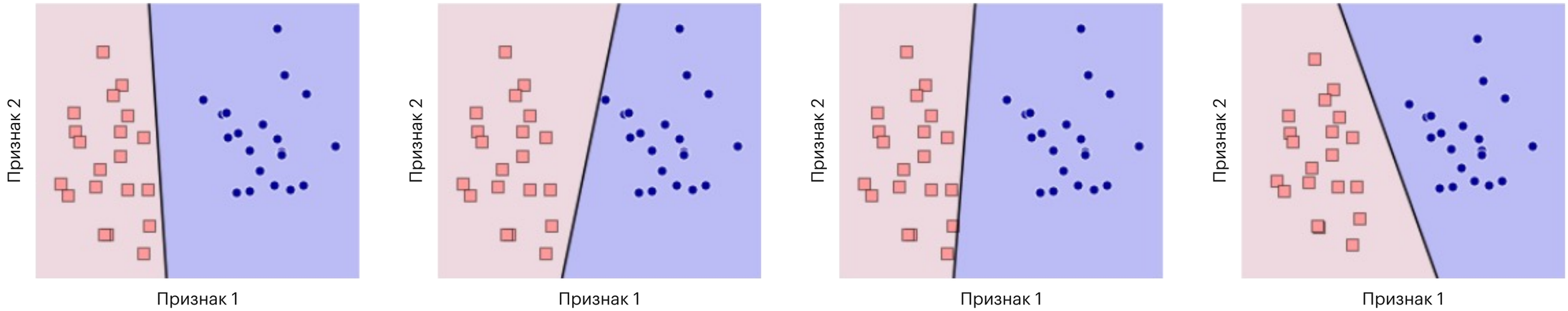
Основное ограничение — линейная разделяющая граница; для задач со сложной структурой данных требуется конструирование дополнительных признаков или переход к более гибким моделям.

Метод опорных векторов (SVM)



Метод опорных векторов

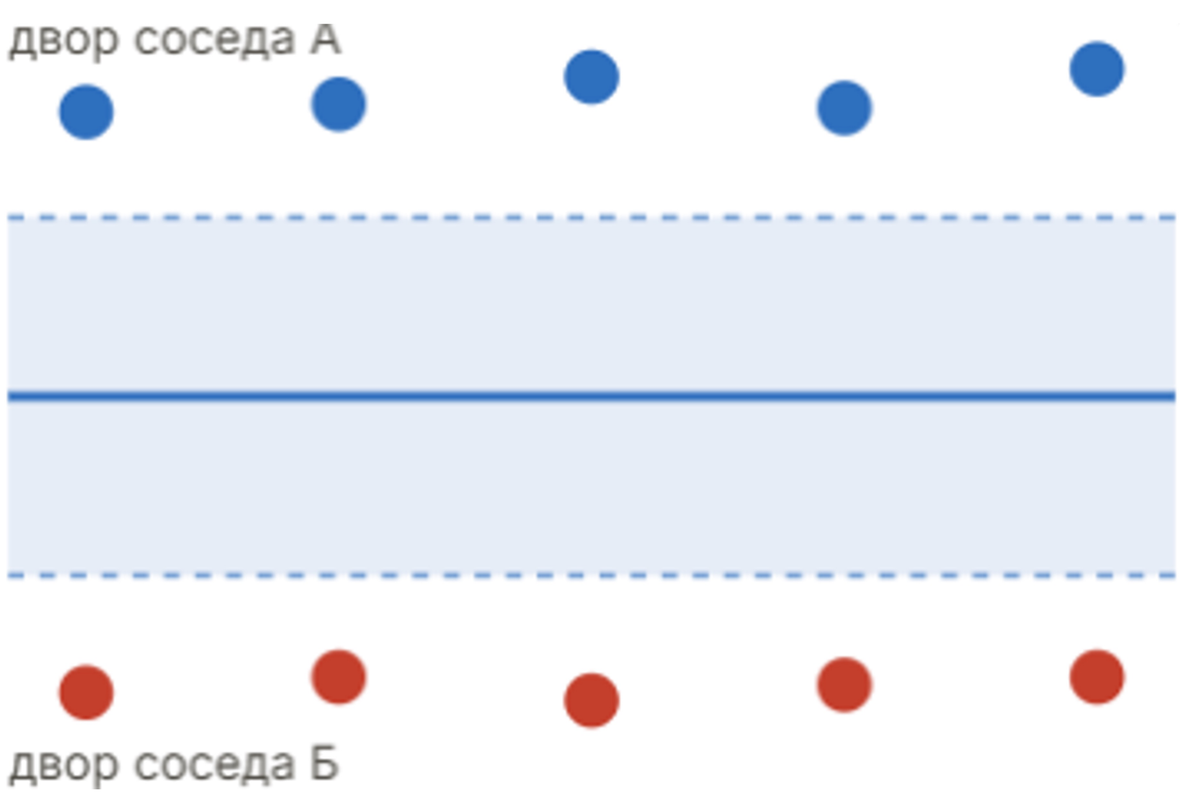
Support Vector Machine (SVM) — это алгоритм классификации, который ищет разделяющую гиперплоскость с максимальным отступом (margin) между классами.



Строительство забора между двух дворов

Если один сосед держит собаку, а другой — курей, забор логичнее ставить не впритык к одному из дворов, а точно посередине нейтральной полосы. Тогда у обоих соседей будет одинаковый запас безопасности.

SVM решает точно такую же задачу: не просто разделить два класса, а провести границу так, чтобы расстояние до ближайших точек обоих классов было максимальным



Метод опорных векторов

Откуда появились «векторы» в названии?



Вектор в SVM — это объект данных, представленный как точка в n-мерном пространстве признаков.

Ключевые элементы

- Каждый объект — вектор признаков.

$$x = (x_1, x_2, \dots, x_n)$$

- Гиперплоскость — линейная граница.

$$w \cdot x + b = 0$$

- Вектор весов w диктует ориентацию границы (перпендикулярен гиперплоскости).
- Параметр b определяет смещение относительно вектора w.

Условия разделения

$$\boldsymbol{w} \cdot \boldsymbol{x}_i + b \geq +1 \text{ для } y_i = +1$$

$$\boldsymbol{w} \cdot \boldsymbol{x}_i + b \leq -1 \text{ для } y_i = -1$$

Метод опорных векторов

Почему векторы «опорные»?

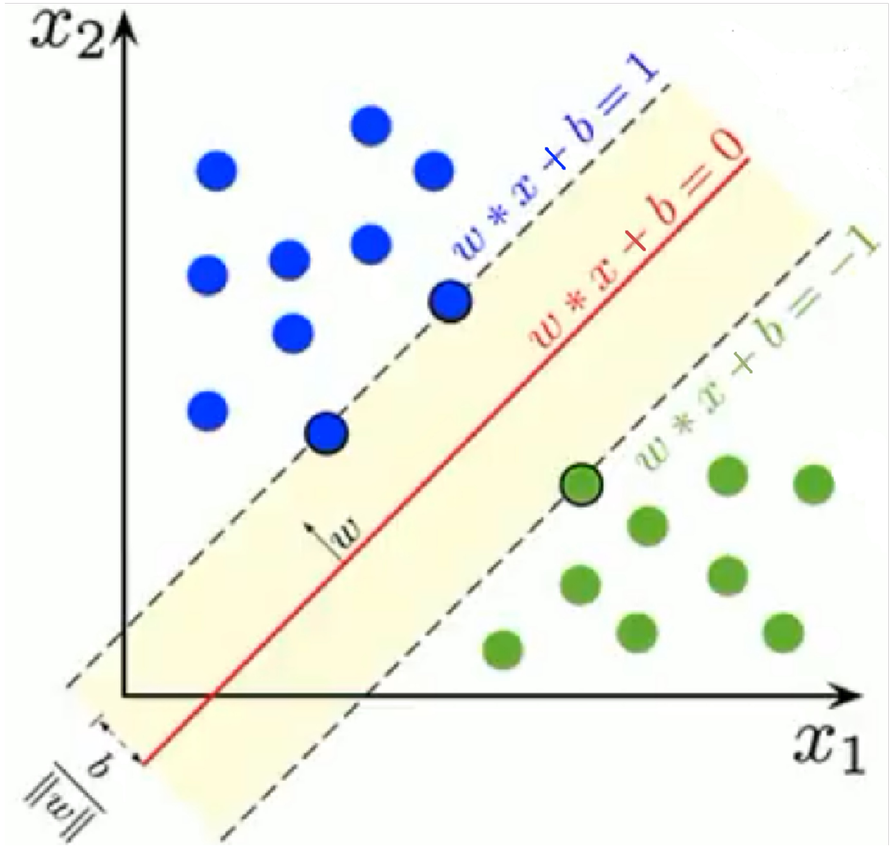
Опорные векторы — это точки, для которых:

- $\langle w, x \rangle + b = 1$ (для класса +1);
- $\langle w, x \rangle + b = -1$ (для класса -1).

Если убрать все точки, кроме опорных векторов, гиперплоскость останется той же. Здесь опорные точки как бы держат границы отступа.

При этом **отступ между крайними точками классов должен быть максимально широким.**

Формализуем определение таких точек.



Математическая постановка

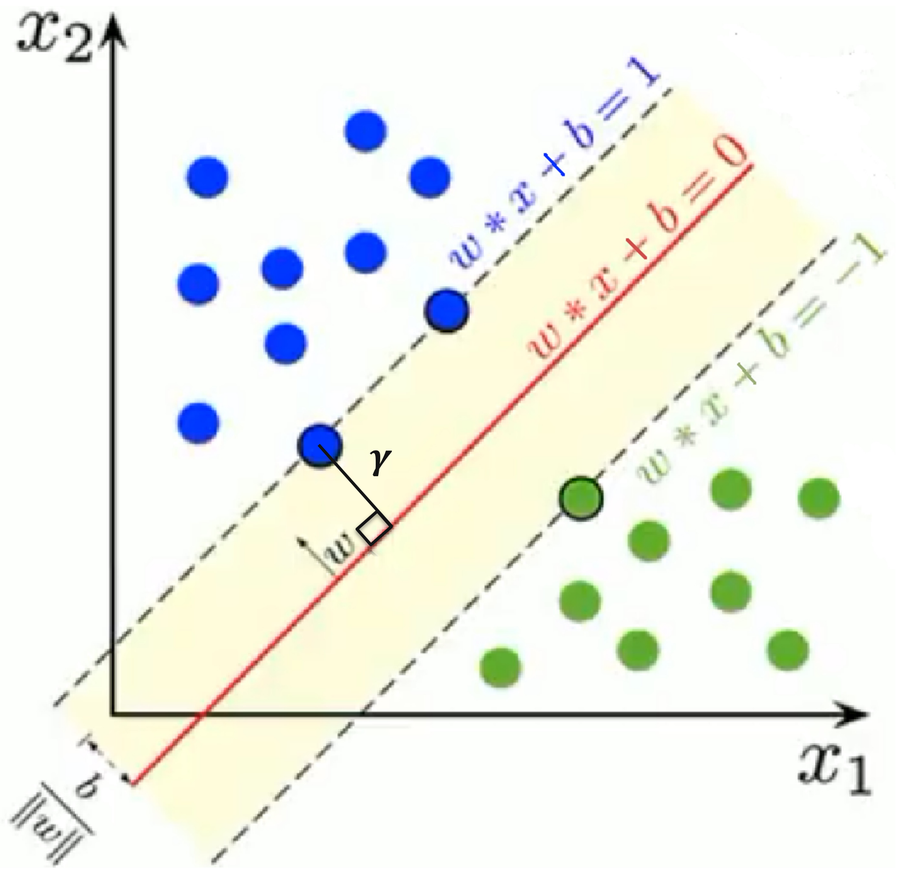
Расстояние между точкой и гиперплоскостью:

$$\text{Расстояние} = \frac{|\langle w, x_i \rangle + b|}{\|w\|}$$

С учетом правильной классификации:

$$\gamma_i = \frac{y_i \cdot (\langle w, x_i \rangle + b)}{\|w\|}$$

Задача: найти минимальное расстояние по всем объектам выборки (i) и максимизировать его.



Какая проблема может возникать при поиске оптимальной разделяющей гиперплоскости?



Математическая постановка

Расстояние между точкой и гиперплоскостью:

$$\text{Расстояние} = \frac{|\langle w, x_i \rangle + b|}{\|w\|}$$

С учетом правильной классификации:

$$\gamma_i = \frac{y_i \cdot (\langle w, x_i \rangle + b)}{\|w\|}$$

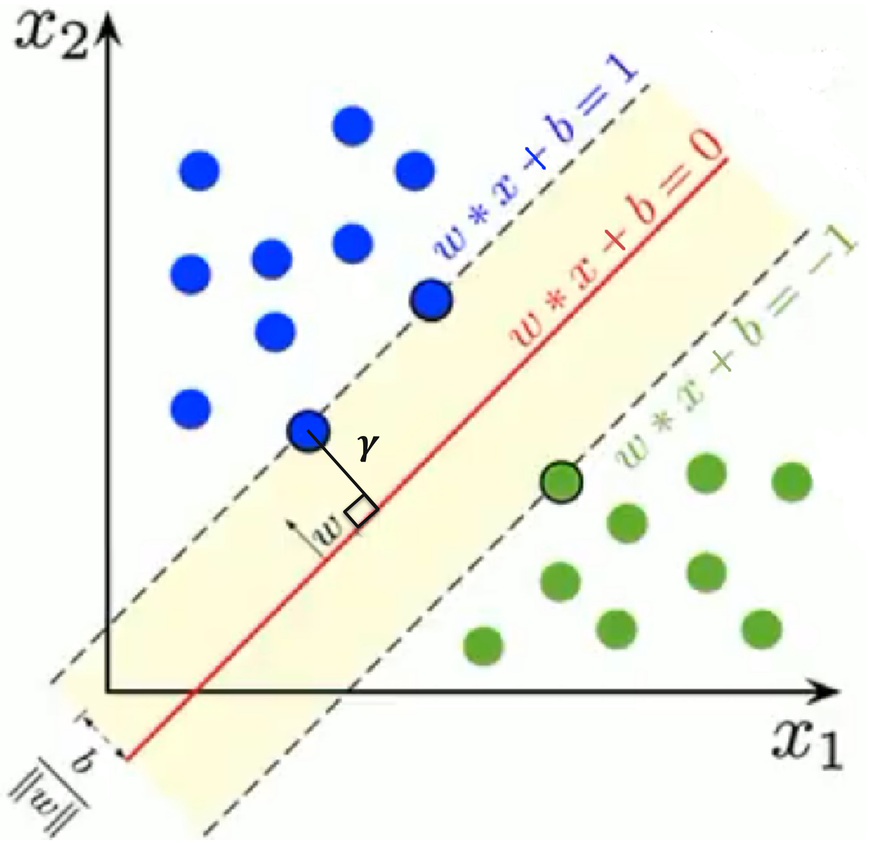
Задача: найти минимальное расстояние
по всем объектам выборки (i) и максимизировать его.

Возникает неоднозначность:

$\langle w, x \rangle + b$ и $\langle cw, x \rangle + cb$ — одна и та же гиперплоскость.

Наложим условие на опорные вектора: $y_i(\langle w, x_i \rangle + b) = 1$

Задача сводится к максимизации: $\gamma = \frac{1}{\|w\|}$



Какая ширина будет у полосы между разделяющей гиперплоскостью и опорными векторами?



Математическая постановка

Вычислим ширину полосы, разделяющей классы

После нормировки ближайшие объекты классов лежат на опорных границах:

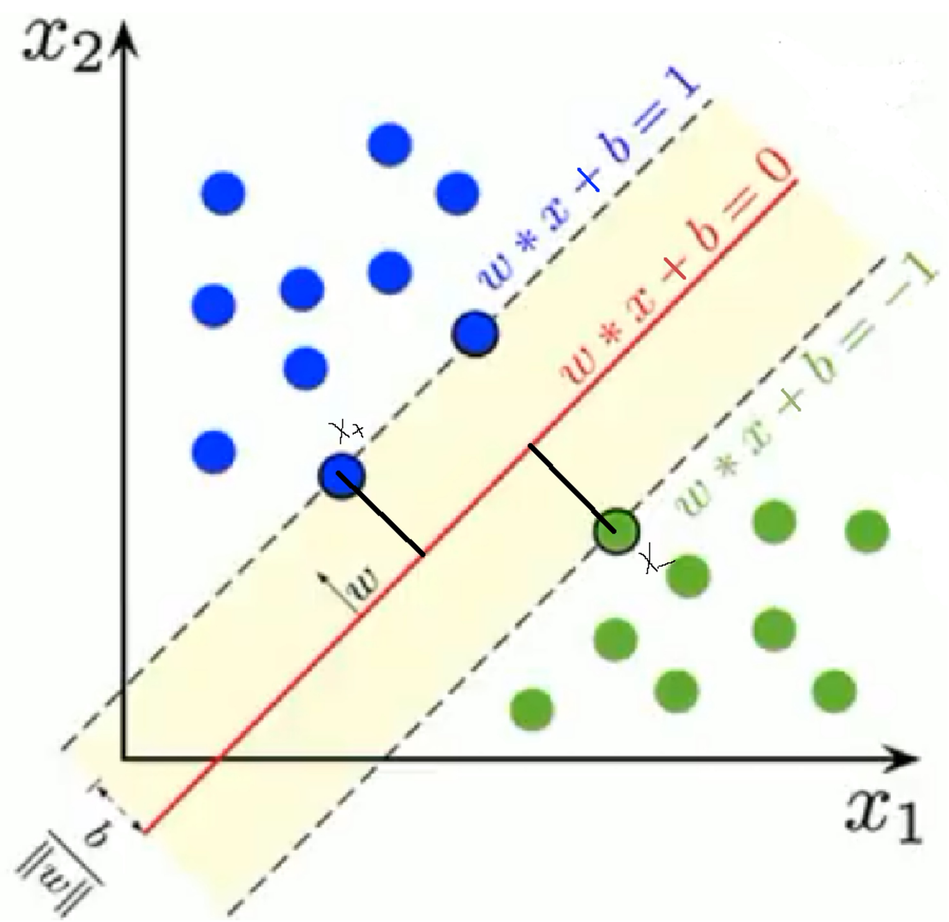
$$\langle w, x\rangle + b = 1, \qquad \langle w, x\rangle + b = -1$$

Расстояние от точки x до разделяющей гиперплоскости $\langle w, x\rangle + b = 0$ равно:

$$\rho(x) = \frac{|\langle w, x\rangle + b|}{\|w\|}$$

Поэтому для показанных на рисунке опорных объектов:

$$\rho(x_+) = \frac{1}{\|w\|}, \qquad \rho(x_-) = \frac{1}{\|w\|}$$



Математическая постановка

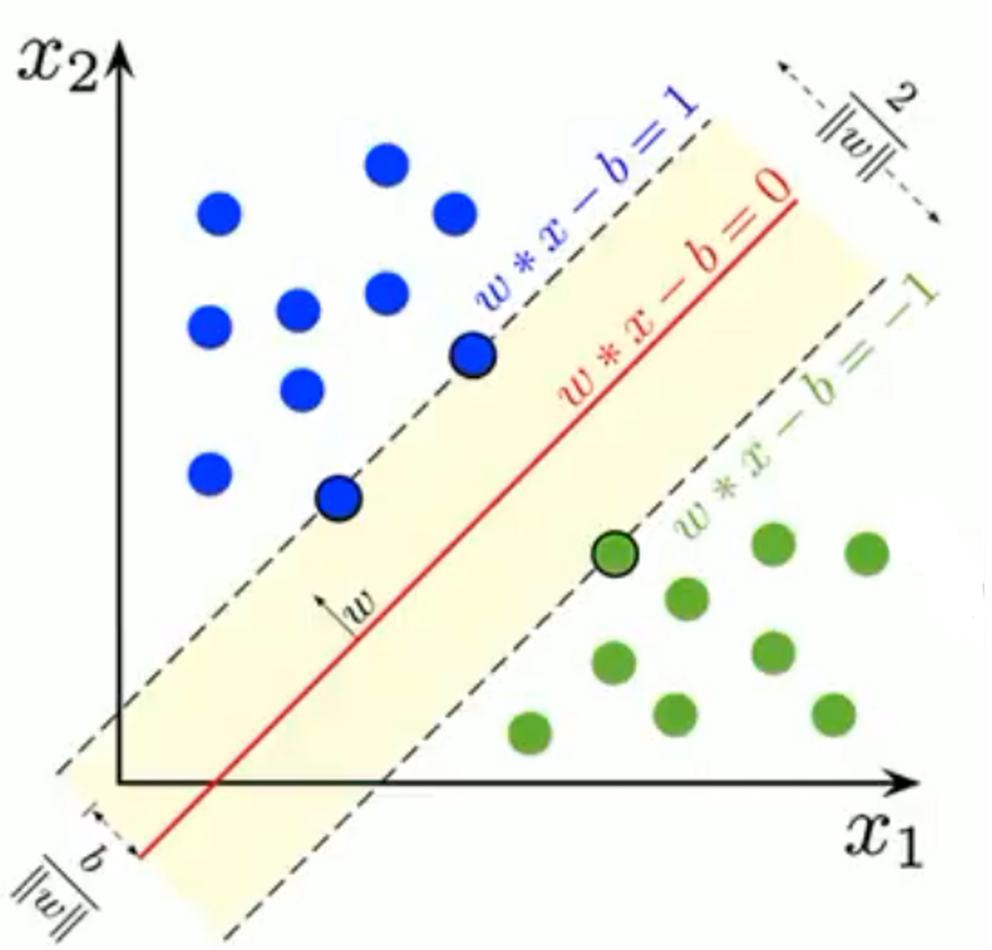
Расстояние от центральной гиперплоскости до каждой опорной границы равно:

$$\frac{1}{\|w\|}.$$

Поэтому полная ширина полосы складывается из двух расстояний:

$$\text{Margin} = \rho(x_+) + \rho(x_-) = \frac{1}{\|w\|} + \frac{1}{\|w\|} = \boxed{\frac{2}{\|w\|}}.$$

Таким образом, чем меньше норма $\|w\|$, тем шире разделяющая полоса.



Ширина зазора и целевая функция SVM

Формула зазора

$$\text{Margin} = \frac{2}{\|\mathbf{w}\|}$$

Цель: максимизируем зазор.

$$\max \frac{2}{\|w\|} \iff \min \|w\| \iff \frac{1}{2}\|w\|^2$$

При условии, что точки классифицированы корректно:

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1, \qquad \forall i.$$

Задача оптимизации

Для линейно разделимого случая

$$\begin{cases} \dfrac{1}{2}\|w\|^2 \to \min\limits_{w,b}, \\ y_i(\langle w, x_i\rangle + b) \geq 1, \quad i = 1, \ldots, \ell. \end{cases}$$

При обработке реальных данных классы часто перекрываются, и идеальная гиперплоскость не существует (линейно неразделимый случай), то есть для любых w и b найдутся точки, нарушающие условие разделения:

$$\exists \mathbf{x}_i \in X : \; y_i(\langle \mathbf{w}, \mathbf{x}_i\rangle + b) < 1.$$

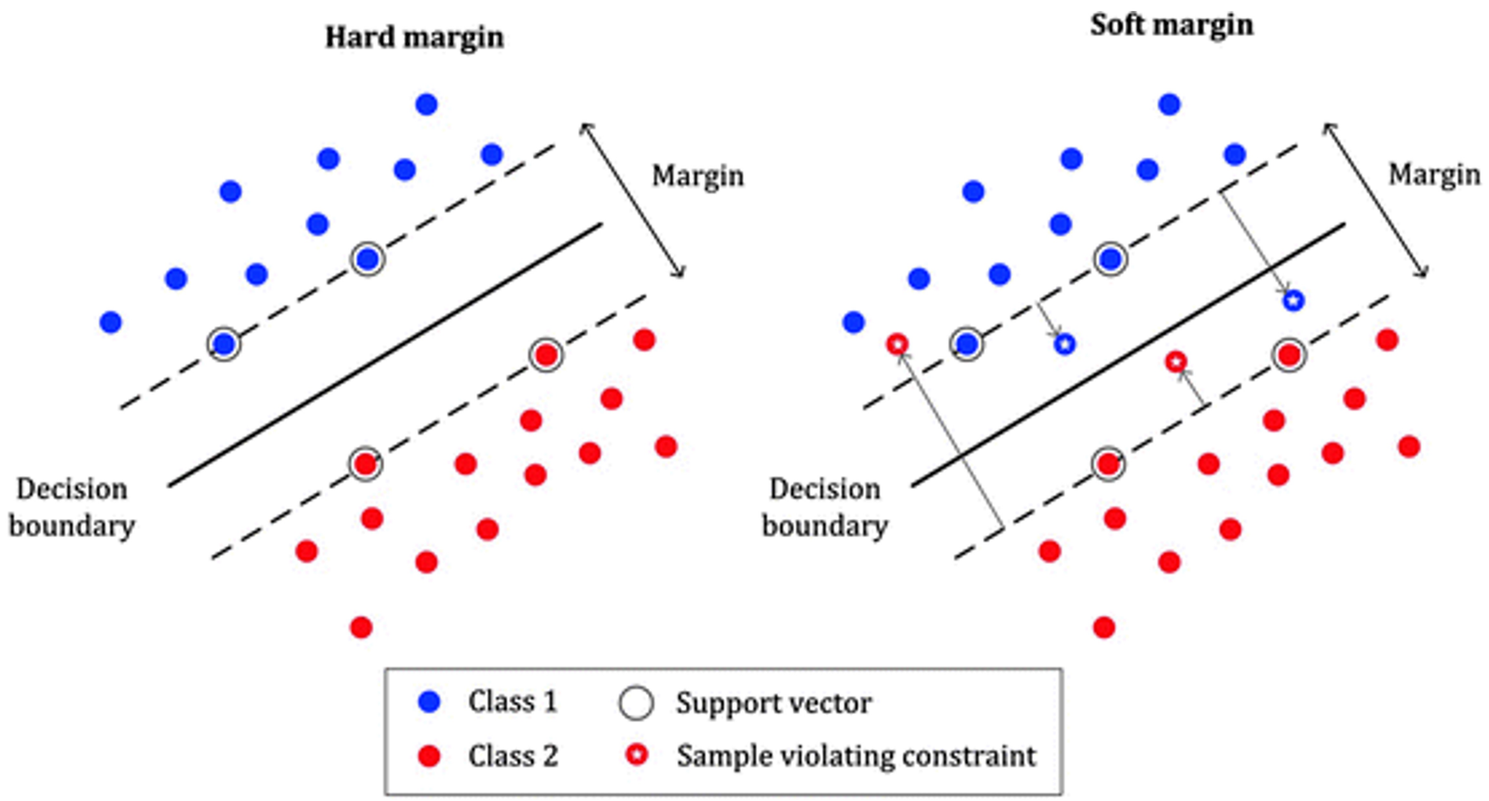
Промежуточный итог



У нас есть работающая задача оптимизации для идеального случая: классы полностью разделимы, и мы просто минимизируем $||w||^2$ при выполнении жёстких условий.

Но что, если классы **неразделимы линейно**?

Сравнение для линейно разделимых и неразделимых данных



SVM для линейно неразделимых данных

Введём переменные, ослабляющие условия.

$$y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \ldots, \ell$$

Новая задача оптимизации

$$\begin{cases} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{\ell} \xi_i \to \min\limits_{\mathbf{w}, b, \xi}, \\ y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1 - \xi_i, \\ \xi_i \geq 0, \quad i = 1, \ldots, \ell. \end{cases}$$

Итог: SVM как баланс между зазором и ошибками.

$$\underbrace{\frac{1}{2}\|\mathbf{w}\|^2}_{\text{Регуляризация (максимизация зазора)}} + C\underbrace{\sum_{i=1}^{n} \max(0, 1 - y_i(\mathbf{w} \cdot \mathbf{x}_i + b))}_{\text{Hinge loss (штраф за ошибки)}} \to \min_{\mathbf{w}, b}$$

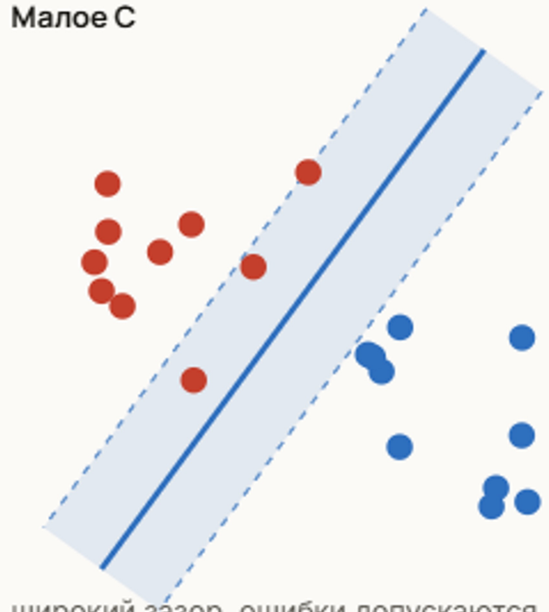
Роль гиперпараметра C в SVM

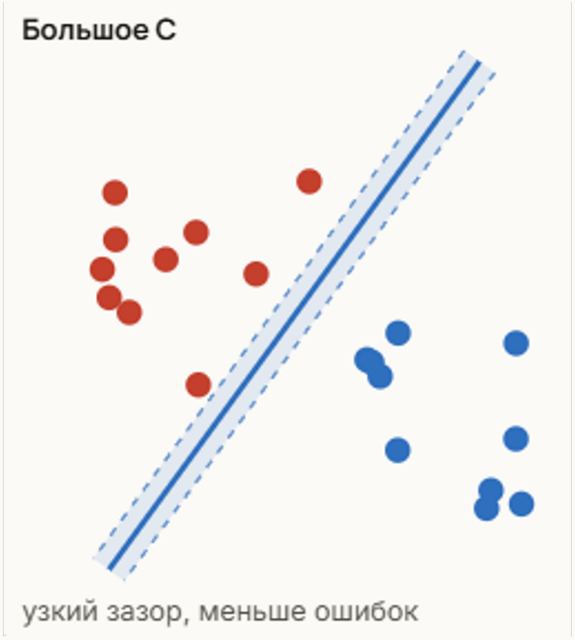
Итог: SVM как баланс между зазором и ошибками.

$$\underbrace{\frac{1}{2}\|\mathbf{w}\|^2}_{\text{Регуляризация (максимизация зазора)}} + \underbrace{C\sum_{i=1}^{n}\max(0, 1 - y_i(\mathbf{w}\cdot\mathbf{x}_i + b))}_{\text{Hinge loss (штраф за ошибки)}} \to \min_{\mathbf{w},b}$$

Малое C — модель прощает больше ошибок, зазор шире, риск недообучения.

Большое C — модель минимизирует нарушения, зазор уже, риск переобучения.





Вызов в sklearn

```
from sklearn.svm import SVC

# Обучаем модель SVM.
model_svm = SVC(
    kernel='linear',
    probability=True,
    class_weight='balanced',
    max_iter=1000,
    C=1.0
)
# kernel — тип ядра.
# probability включает расчёт вероятностей через калибровку по Платту; удобно, если хотим получить
predict_proba,
но делает обучение медленнее.
# class_weight учитывает дисбаланс классов.
# max_iter — число итераций до сходимости; нужно увеличивать при предупреждении ConvergenceWarning.
# C — коэффициент штрафа за нарушения отступа. Чем больше C, тем сильнее модель штрафуется за
нарушения и тем слабее регуляризация. Большое C обычно даёт более узкий зазор и повышает риск
переобучения.
```

Вызов в sklearn

```
# Фит на данные
model_svm.fit(X_train, y_train)

# Предсказания вероятностей принадлежности к положительному классу (1)
y_pred_prob = model_svm.predict_proba(X_test)[:, 1]

# Предсказания классов
y_pred_classes = model_svm.predict(X_test)

# Доступ к коэффициентам модели (только для линейного ядра!)
print(model_svm.coef_)
print(model_svm.intercept_)
```

Промежуточный итог



Теперь у нас есть версия SVM, устойчивая к выбросам и умеющая настраивать компромисс между шириной полосы и числом ошибок через C.

Но данные могут быть устроены так, что в исходном признаковом пространстве разделяющая гиперплоскость будет допускать слишком много ошибок.

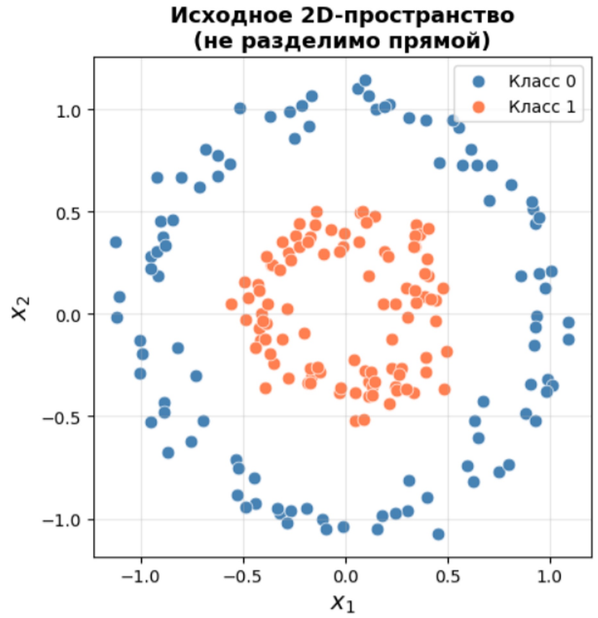
Ядерный трюк (Kernel Trick)

Проблема:

из-за нелинейно разделимых данных в исходном пространстве невозможно построить линейную границу.

Идея:

подставлять функцию $K(x_i, x_j)$, которая ведёт себя так, как будто мы работаем в пространстве высокой размерности, — это и есть ядерный трюк.



Решение:

ядровая функция заменяет явное отображение $\varphi(x)$, позволяя обучать линейную модель в «нелинейном» пространстве.

$$K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$$

Переход к двойственной задаче

Чтобы применить ядерный трюк, нужно перейти к двойственной задаче. Для этого выписываем Лагранжиан исходной задачи для линейно неразделимого случая:

$$\mathcal{L} = \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{\ell}\xi_i - \sum_{i=1}^{\ell}\alpha_i\left[y_i(\langle \mathbf{w}, \mathbf{x}_i\rangle + b) - 1 + \xi_i\right] - \sum_{i=1}^{\ell}\mu_i\,\xi_i$$

Ищем его экстремумы по исходным переменным и получаем двойственную задачу:

$$\begin{cases} \sum_{i=1}^{\ell}\alpha_i - \frac{1}{2}\sum_{i=1}^{\ell}\sum_{j=1}^{\ell}\alpha_i\,\alpha_j y_i y_j\langle x_i, x_j\rangle \to \max, \\ \sum_{i=1}^{\ell}\alpha_i\,y_i = 0, \\ 0 \le \alpha_i \le C. \end{cases}$$

Ключевая идея: оптимизация зависит только от попарных скалярных произведений между объектами обучающей выборки. Скалярное произведение можно заменить допустимой ядерной функцией $K(x_i, x_j)$, соответствующей скалярному произведению объектов в некотором пространстве признаков.

Промежуточный итог



Теперь у нас есть двойственная форма SVM, которая зависит от скалярных произведений — к ним можно применять различные ядра.

Полиномиальное ядро (Polynomial Kernel)

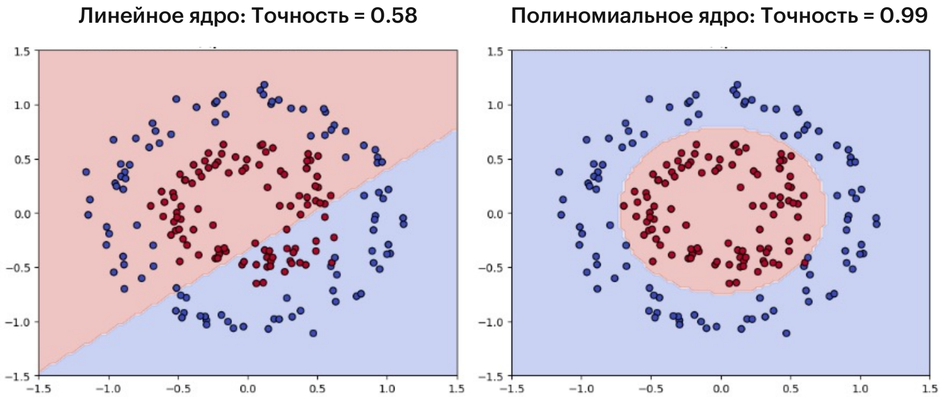
Оно использует полиномиальные функции для отображения данных, подходит для задач, где граница между классами имеет полиномиальную форму (кольца, спирали).

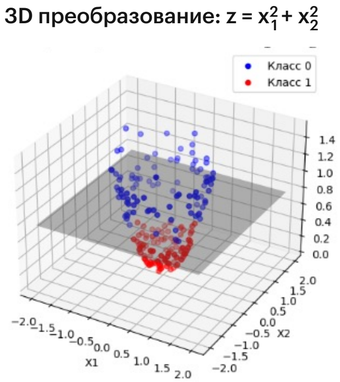
Формула

$$K(x_i, x_j) = (\langle x_i, x_j \rangle + c)^d$$

Пример признаков (при d = 2)

$$\phi(x) = \left[1, x_1, x_2, x_1^2, x_2^2, x_1 x_2\right]$$





Вызов в sklearn

```
from sklearn.svm import SVC

# Обучаем модель SVM.
model_poly = SVC(kernel='poly', degree=3, coef0=1)

# kernel — используемое ядро; 'poly' для полиномиального ядра.
# degree — степень полинома d; больше степень → сложнее граница, выше риск переобучения.
# coef0=1 — сдвиг C в функции ядра; влияет на форму полинома (особенно при малых x_i · x_j),
полезен для настройки гибкости.
```

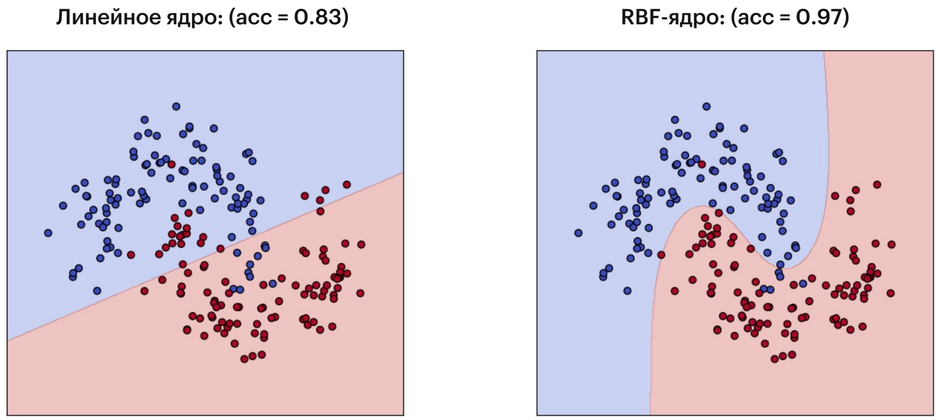
Гауссово (RBF) ядро

Использует радиальную базисную функцию, подходит для задач с неочевидной структурой (например, кластеров сложной формы).

Формула

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

γ — гиперпараметр.
Чем он больше, тем сложнее граница.



Вызов в sklearn

```
from sklearn.svm import SVC

# Обучаем модель SVM с RBF-ядром.
model_rbf = SVC(kernel='rbf', gamma='scale')

# kernel — используемое ядро; 'rbf' для гауссова ядра.
# gamma — параметр γ; управляет «радиусом внимания» к объектам.
# маленькое gamma → гладкая граница, учитываются дальние точки.
# большое gamma → граница плотно обвивает обучающие объекты, возможен оверфит (переобучение).
# gamma='scale' автоматически подбирает gamma по встроенной формуле.
```

Применение



SVM важен для истории машинного обучения тем, что развил принципы по работе с ядрами и максимизации разделяющего зазора.

Со временем, эти принципы легли в основу современных алгоритмов:

- ядра используются в методах kernel trick — например, kernel PCA, Kernel Ridge Regression;
- максимизация зазора вдохновила подходы к регуляризации и margin-based learning — например, в бустинге и некоторых нейронных сетях.

Сейчас метод применяется крайне редко из-за масштабируемости (сложность $O(n^2)$ по количеству данных) и скорости современных алгоритмов.

Итоги SVM

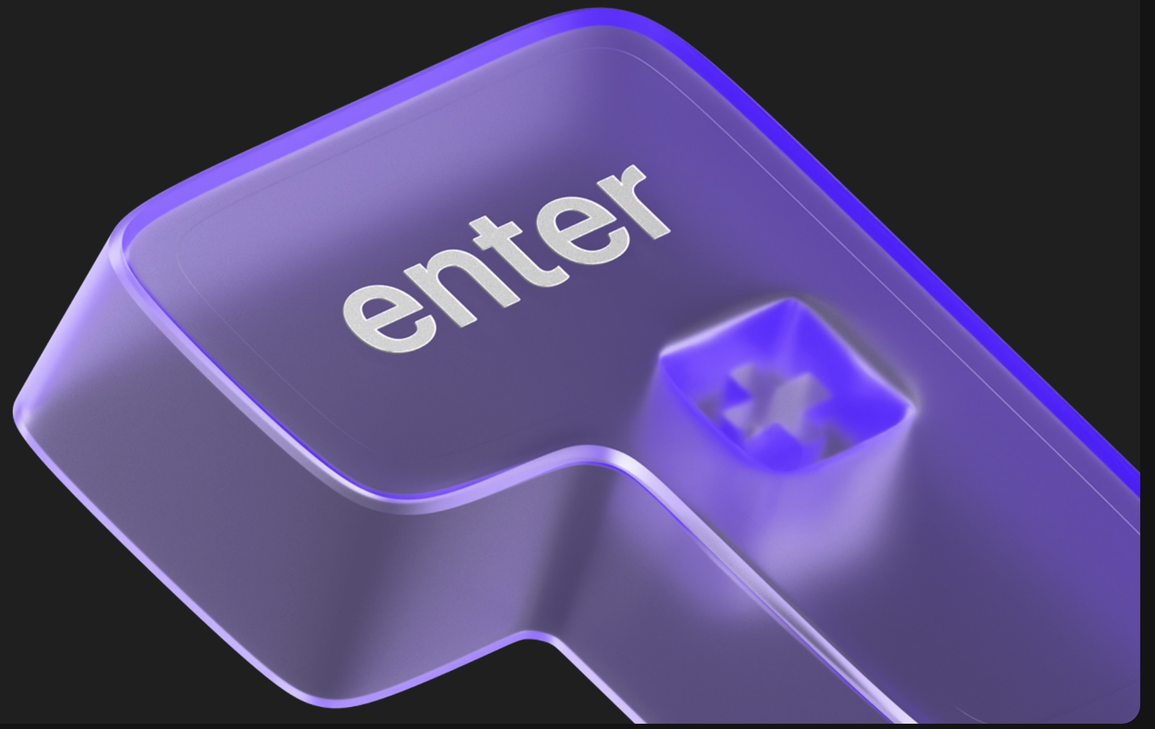
SVM — модель бинарной классификации, которая строит линейную разделяющую гиперплоскость, равноудаленную от обоих классов.

Обучение модели основано на поиске максимально широкой полосы между двумя классами.
Если такую полосу провести невозможно, то допускается совершение ошибок.

Модель идейно проста и интерпретируема, но тяжела для вычислений. Поэтому, если и используется, то на малых данных.

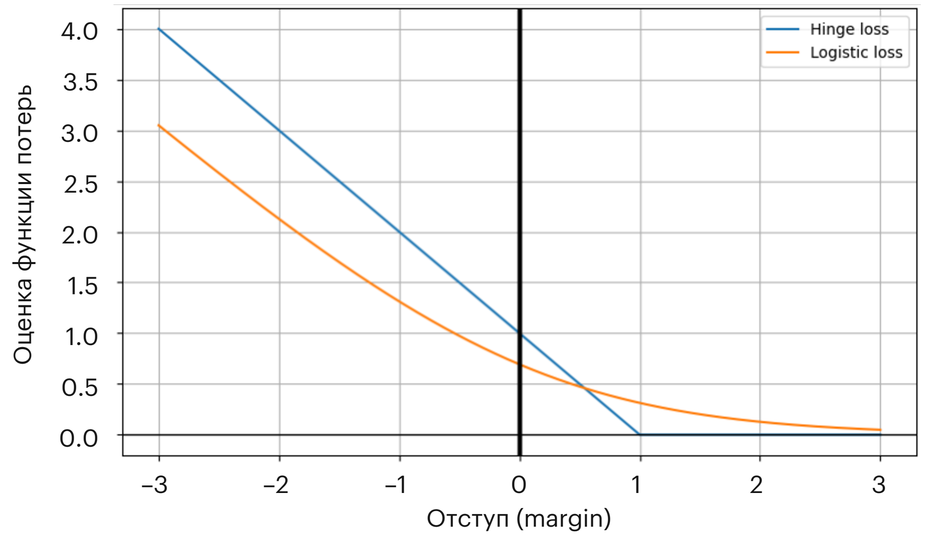
Ядерный трюк помогает эффективно использовать нелинейные признаки — без дополнительных затрат на вычисление.

Отличия SVM
и логистической регрессии

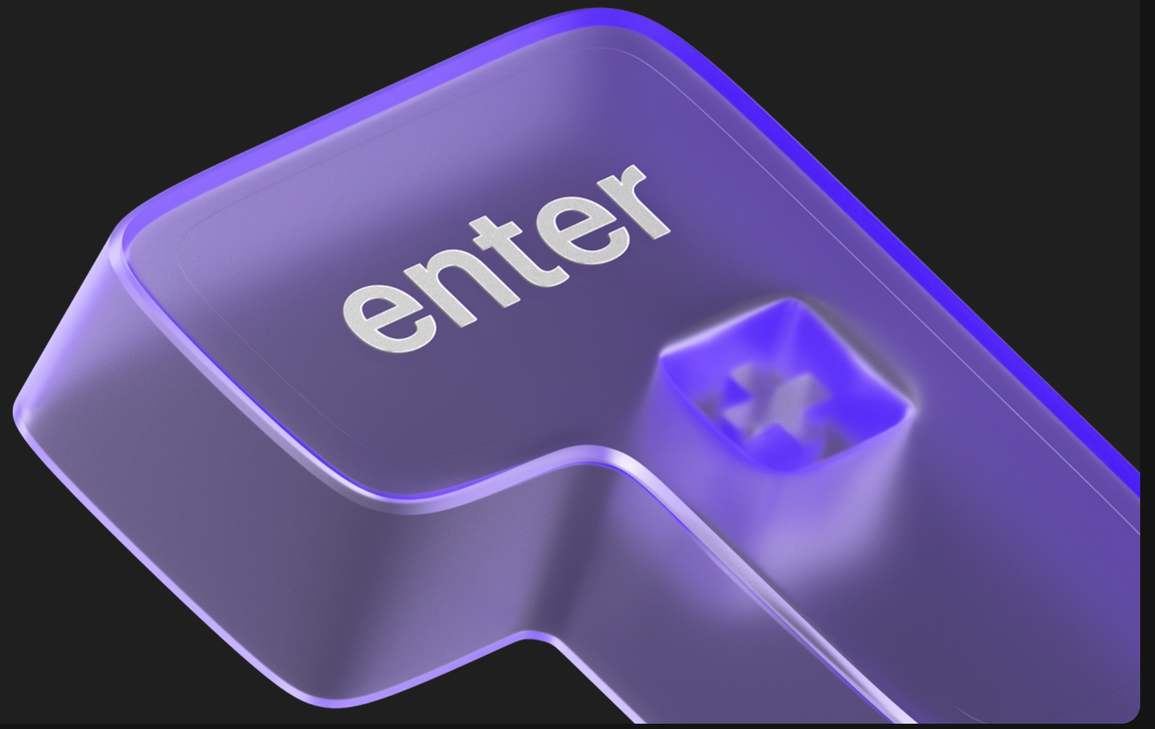


Отличия SVM и логистической регрессии

Критерий	SVM	Логистическая регрессия
Интерпретации	Геометрические: опорные векторы и ширина зазора; веса указывают направление и важность признаков	Вероятностные: можно оценивать риски, настраивать порог принятия решений
Чувствительность к выбросам	Менее чувствителен к далёким выбросам	Выбросы могут сильно смещать границу
Работа с большими выборками	Вычислительно сложнее (особенно с нелинейными ядрами), плохо масштабируется на миллионы объектов	Быстрее обучается, хорошо масштабируется, имеет эффективные солверы для больших данных
Вывод	Решающее правило	Вероятности
Использование	Редко в практике	Широко



Калибровка вероятностей



Калибровка вероятностей

Проблема:

модели не всегда выдают «настоящие» вероятности → может быть переоценка или недооценка риска.

Решение:

калибровка — приведение предсказанных вероятностей в соответствии
с реальной вероятностью события.

Как оценить? Калибровочная кривая:

X — средняя предсказанная вероятность в бине;

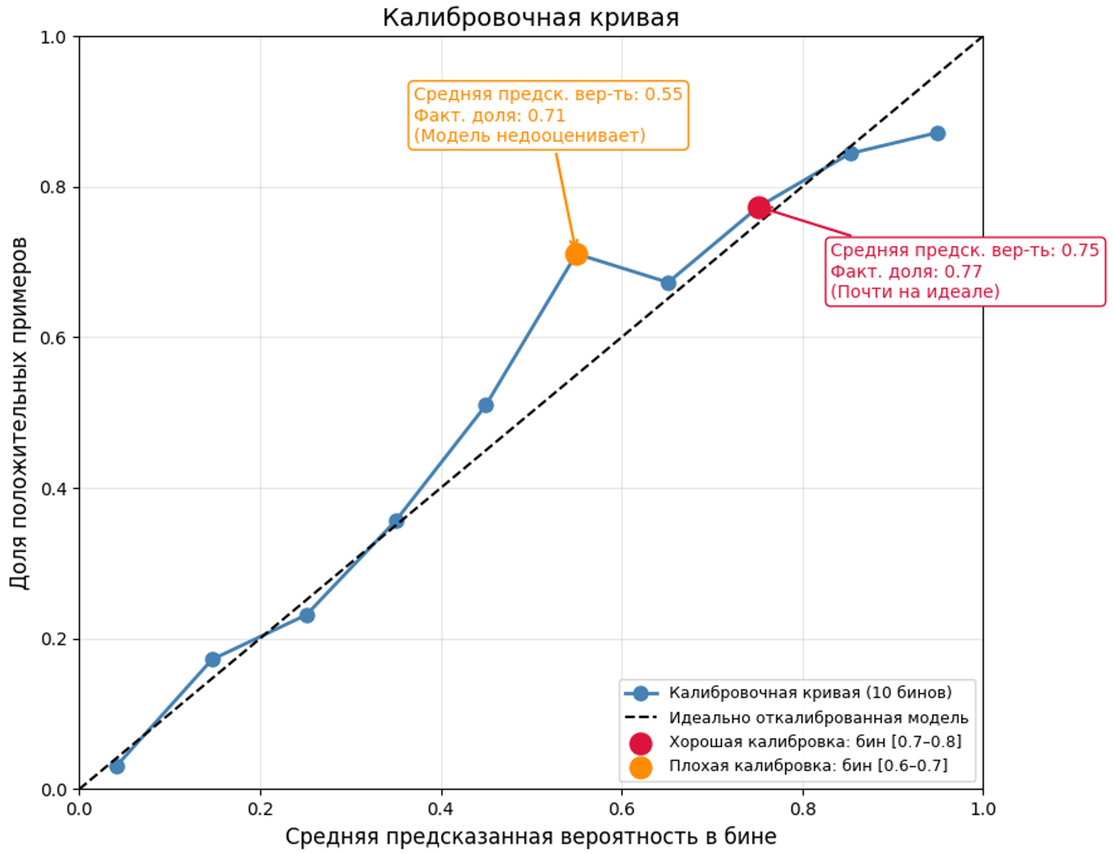
Y — фактическая доля положительных классов в бине;

идеал — точки на линии $y = x$.

Калибровочная кривая

- **Диапазон 0.7 - 0.8 (красная точка)**
  Модель предсказала среднюю вероятность в 75%. Событие действительно произошло в 77% случаев. Модель работает почти идеально.

- **Диапазон 0.5 - 0.6 (оранжевая точка)**
  Была предсказана вероятность 55%, однако на самом деле событие наступило в 71% случаев. Модель недооценила вероятность.



Сигмоидная калибровка Платта (Platt Scaling)

Используется для моделей без естественной вероятностной интерпретации (например, SVM или деревьев).

Суть метода — подгонка логистической функции к выходам модели.

Формула

$$P(y = 1 \mid x) = \frac{1}{1 + \exp(Af(x) + B)}$$

```
from sklearn.calibration import
CalibratedClassifierCV

calibrated_model = CalibratedClassifierCV(
    estimator=base_model,
    method="sigmoid",
    cv=5,
)
```

Изотоническая регрессия

Строит произвольную монотонную функцию без заданной заранее формы.

Обязательное требование: функция должна быть монотонно неубывающей.

Результат — ступенчатая функция.

Формула

$$\min_{g \text{ монотонна}} \sum_{i=1}^{n} \left(g(f(x_i)) - y_i\right)^2$$

```
from sklearn.calibration import
CalibratedClassifierCV

calibrated_model = CalibratedClassifierCV(
    estimator=base_model,
    method="isotonic",
    cv=5,
)
```

Метрики оценки

Критерий	Log Loss	Brier Score
Суть метрики	Оценивает, насколько вероятности близки к истине	MSE между предсказанной вероятностью и фактом
Формула	$\text{LogLoss} = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(p_i) + (1-y_i)\log(1-p_i)]$	$\text{Brier} = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)^2$
Чувствительность к ошибкам	Экспоненциальный рост штрафа при ошибках с высокой уверенностью	Квадратичный штраф, ограниченный значениями от 0 до 1
Реакция на экстремальные ошибки	Очень высокая	Умеренная
Ключевое преимущество	Прямо указывает на проблемы калибровки	Устойчива к выбросам
Когда использовать?	Если точность вероятностей важна (например, в медицине или финансах)	Для общей оценки качества вероятностей

Вызов в sklearn

```
from sklearn.svm import SVC
from sklearn.calibration import CalibratedClassifierCV
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,
    random_state=42,
)

base_model = SVC(kernel="poly", degree=3)

calibrated_model = CalibratedClassifierCV(
    estimator=base_model,
    method="sigmoid",
    cv=5,
)

calibrated_model.fit(X_train, y_train)

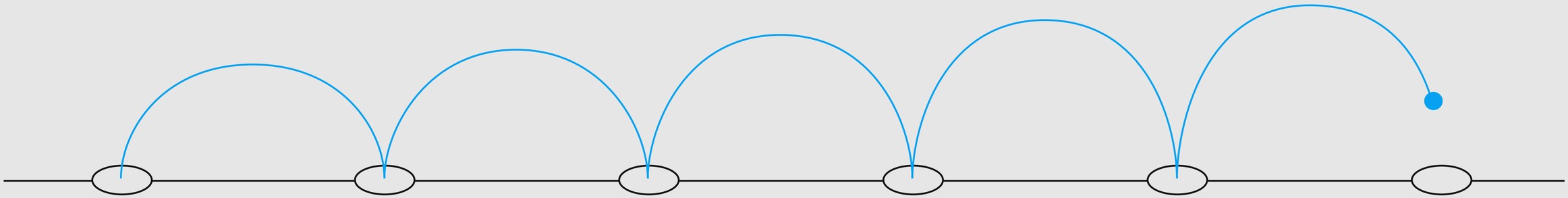
y_pred_prob = calibrated_model.predict_proba(X_test)[:, 1]
```

Применение на реальных данных

Используем **Default of Credit Card Clients Dataset** в Kaggle.

Задача: спрогнозировать дефолт по кредитной карте.

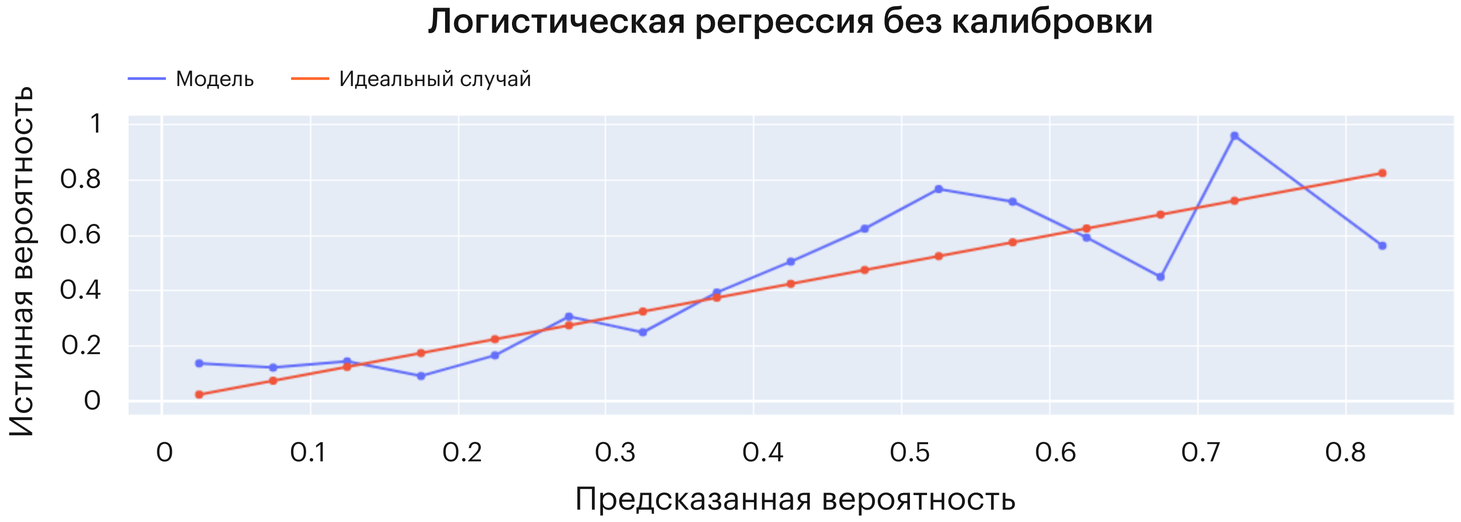
Модель	Brier Score	Log Loss	AUC-ROC
Логистическая регрессия	0.146	0.470	0.719
Изотоническая калибровка	0.140	0.449	0.724
Разница (%)	-4.1%	-4.5%	+0.7%

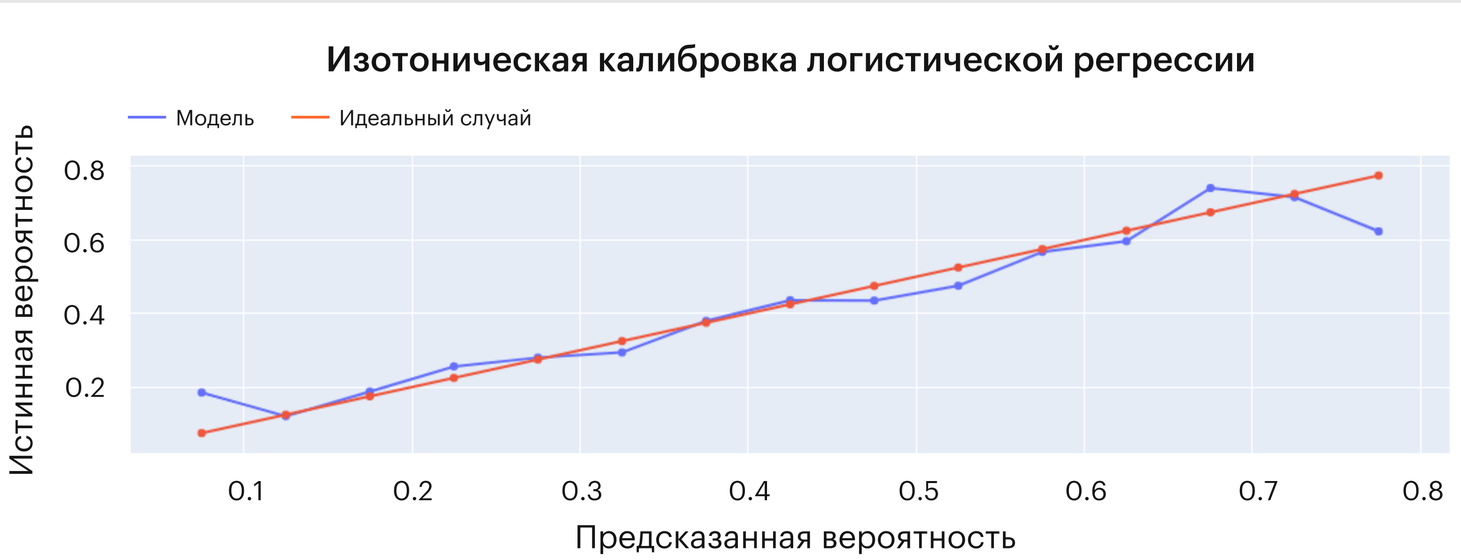


Применение на реальных данных

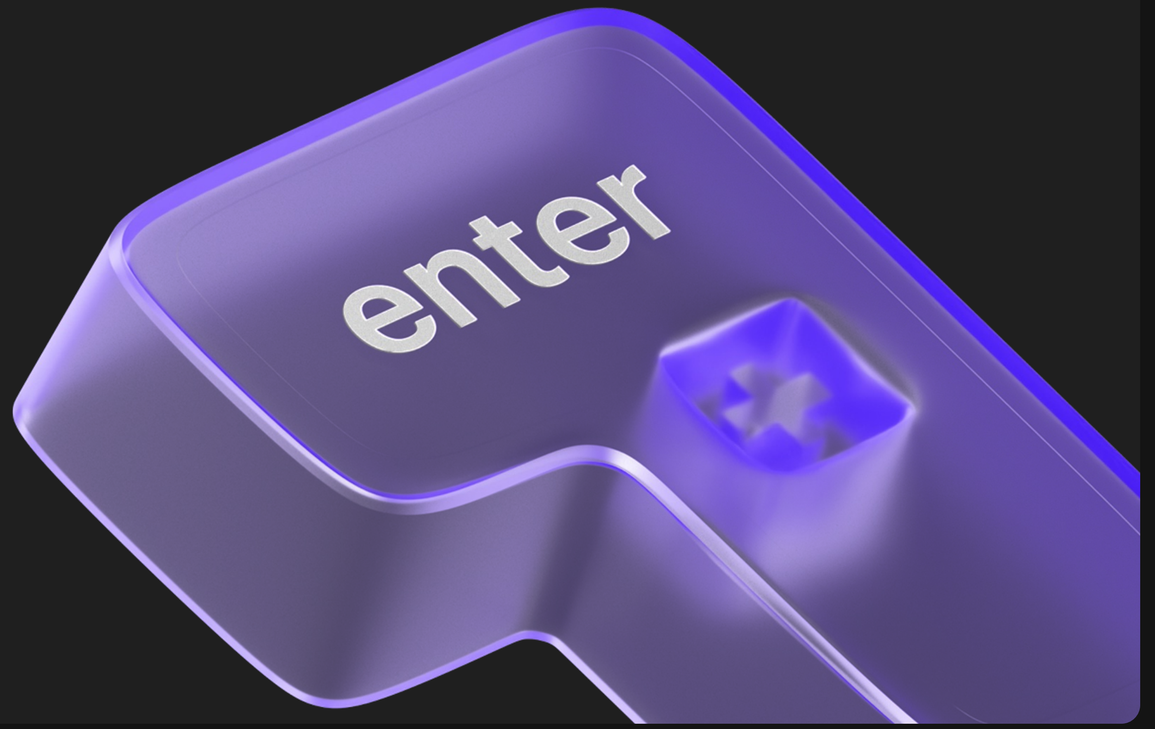
Вывод: мы улучшили качество вероятностных предсказаний (Brier, LogLoss), AUC почти не изменился.

Это отличный результат, поскольку важна точность: дефолт по карте — это финансовые потери.





Преобразование признаков и целевой переменной



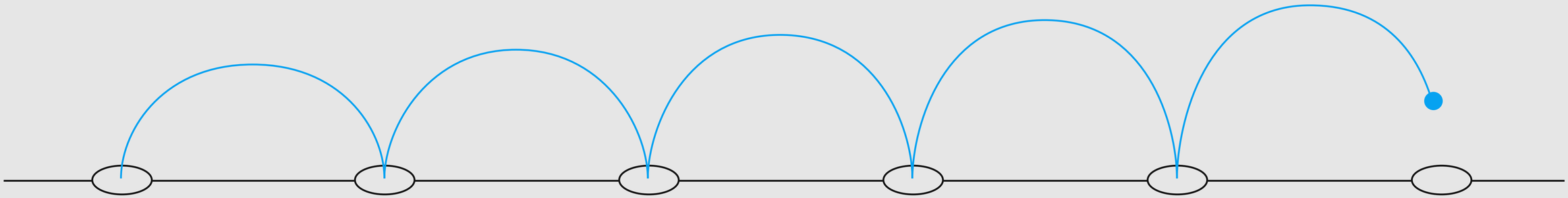
Зачем нужны преобразования

У целевой переменной может быть неудобное распределение:

- сильно смещённое (асимметричное);
- с выбросами или длинным хвостом (например, логнормальное или экспоненциальное).

Преобразование помогает:

- улучшить качество модели (особенно в регрессии);
- стабилизировать дисперсию и сделать распределение ближе к нормальному.



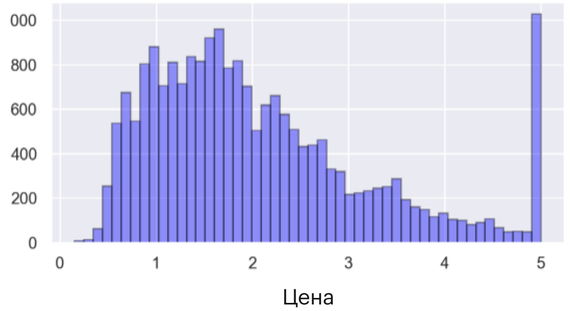
Методы, изменяющие масштаб

1. Логарифмирование

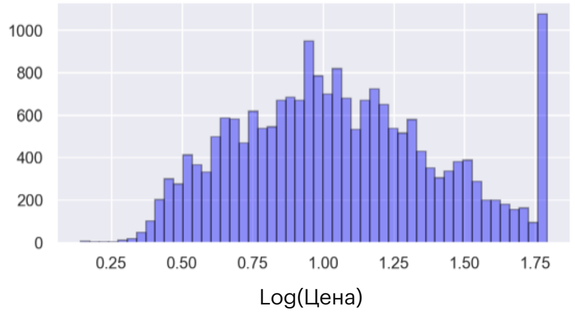
Формула: $y_{\text{new}} = ln(y)$

Для данных с тяжелыми хвостами (например, финансовые датасеты — акции, недвижимость).

До преобразования



После логарифмирования

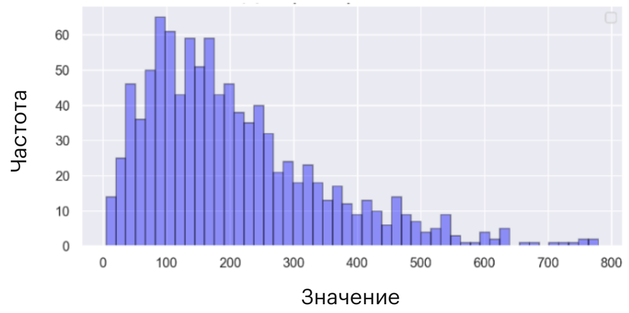


2. Корень квадратный/кубический

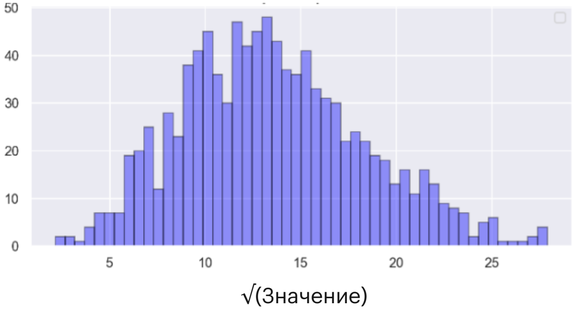
Формула: $y_{\text{new}} = \sqrt{y}$ или $y_{\text{new}} = \sqrt[3]{y}$.

Для данных с пуассоновским распределением (например, количество событий — клики на сайте).

До преобразования



После преобразования



Методы, изменяющие постановку задачи

3. Бинаризация

Формула: $y_{\text{new}} = \begin{cases} 1, & \text{если } y \geq \text{порог}, \\ 0, & \text{иначе}. \end{cases}$

Приведение к бинарной задаче (например, поиск аномалий — сверхвысоких зарплат).

4. Ранжирование

Заменяет значения y на их порядковые номера в отсортированном списке.

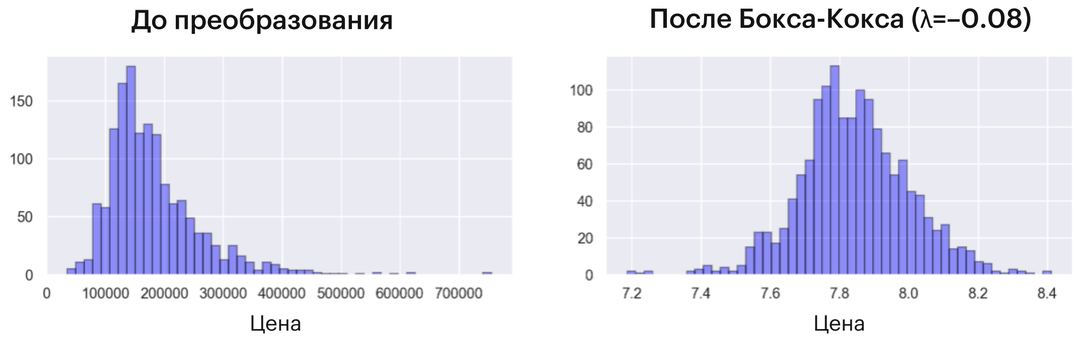
Полезно, если:

- необходимо снизить влияние выбросов и масштабов (например, экстремально высокие доходы не искажают модель, поскольку учитывается лишь порядок).
- важен порядок, а не сами значения (например, владельцу интернет-магазина важнее знать, какие товары будут в топе, середине или внизу рейтинга, чем точные объёмы продаж).

Методы, подбирающие форму преобразования

5. Преобразование Бокс-Кокса (Box-Cox)

Формула: $$y_{\text{new}} = \begin{cases} \dfrac{y^{\lambda} - 1}{\lambda}, & \text{если } \lambda \neq 0, \\ \log(y), & \text{если } \lambda = 0. \end{cases}$$



- Делает данные ближе к нормальному распределению.
- Выравнивает дисперсию.
- Помогает при гетероскедастичности (когда ошибки модели растут с увеличением y).

Например, если мы изучаем прогноз спроса, Box-Cox сгладит масштаб:

- популярные товары (10–1000 продаж в месяц);
- нишевые (1–10 продаж в месяц).

Ограничение: работает только с положительными y.

Методы для отрицательных значений и выбросов

6. Трансформация Йео-Джонсона (Yeo-Johnson)

Формула:

$$y_{\text{new}} = \begin{cases} \dfrac{(y+1)^{\lambda}-1}{\lambda}, & y \geq 0,\ \lambda \neq 0, \\ \log(y+1), & y \geq 0,\ \lambda = 0, \\ -\dfrac{(1-y)^{2-\lambda}-1}{2-\lambda}, & y < 0,\ \lambda \neq 2, \\ -\log(1-y), & y < 0,\ \lambda = 2. \end{cases}$$

Аналог Box-Cox. Может быть менее стабилен, но работает с нулями / отрицательными y (например, предсказание прибыли — дохода или убытка).

7. Обрезка выбросов по перцентилям

- Значения ниже нижнего перцентиля заменяются нижней границей, а значения выше верхнего перцентиля — верхней границей
- Благодаря этому снижается вред от выбросов при сохранении основной структуры данных (например, для одной должности могут указать зарплату от 1 рубля до миллиарда — обрезка убирает экстремумы и делает задачу адекватной).

Итоги

Логистическая регрессия:

- функция потерь — «штрафование» за уверенные ошибки;
- даёт честную вероятность: предсказание 0.8 → среди таких объектов примерно 80% действительно относятся к положительному классу.

SVM (метод опорных векторов):

- ищет границу с максимальным зазором между классами;
- использует опорные объекты (ближайшие к границе);
- не выдаёт честных вероятностей (расстояние до границы ≠ вероятность);
- справляется с выбросами лучше, чем логистическая регрессия.

Калибровка вероятностей:

- проверяет, насколько модель честна в вероятностях;
- позволяет получить вероятности для моделей, если их нет (SVM, деревья).

Методы:

- **сигмоида Платта** — логистическая кривая, даёт гладкое приближение;
- **изотоническая регрессия** строит монотонную функцию, даёт ступенчатый результат. Риск переобучения.

Преобразование признаков и целевой переменной

Цель — справиться с асимметрией, выбросами и тяжёлыми хвостами.



ЦЕНТРАЛЬНЫЙ УНИВЕРСИТЕТ



Вопросы